BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE

B.Sc. Computer Science · Capstone Project · Academic Year 2025–2026

FINAL PROJECT REPORT

Solana Local Explorer

A self-hosted, one-command block explorer and real-time transaction indexer for private Solana validators, built on the Yellowstone gRPC Geyser interface.

SUBMITTED BY

Garvit Dadheech · Shubh Kesharwani

UNDER THE GUIDANCE OF

Dharma Teja

Project Supervisor

`solana-test-validator 1.18.26` · `Yellowstone gRPC Geyser 1.15.3` · `TypeScript` · `Prisma` · `PostgreSQL 15` · `Next.js 16` · `Docker Compose`

Submitted in partial fulfilment of the requirements for the degree of B.Sc. Computer Science · 28 August 2026

CERTIFICATE OF APPROVAL

TO BE SIGNED BY THE PROJECT SUPERVISOR

This is to certify that the project report entitled **"Solana Local Explorer — A Self-Hosted Block Explorer and Real-Time Transaction Indexer for Private Solana Validators"** is a bona fide record of the capstone project work carried out by **Garvit Dadheech** and **Shubh Kesharwani**, students of B.Sc. Computer Science (2025–2026), under my supervision and guidance.

The work presented in this report is original, has been carried out by the candidates themselves, and has not been submitted elsewhere for the award of any other degree or diploma. The system described herein has been demonstrated to me in working condition, and the accompanying source code, test suite and documentation have been reviewed.

The report is hereby approved for submission and for the conduct of the project viva voce examination.

Dharma Teja
Project Supervisor

Date: _____

Viva Voce Examiner

Date: _____

PROJECT ARTEFACTS REVIEWED

Source repository (GitHub) · source archive (ZIP) · this final report · project summary · user manual · installation guide · test cases and validation report · final presentation deck · recorded demonstration.

DECLARATION OF ORIGINALITY

We hereby declare that the project report entitled **"Solana Local Explorer"** submitted in partial fulfilment of the requirements for the degree of B.Sc. Computer Science is our own work. All external sources — open-source software, technical documentation, specifications and prior literature — that have been consulted or incorporated are explicitly acknowledged in the References section and, where code is reused, in the corresponding source files.

In particular, we acknowledge that the project builds on the following third-party open-source components, which are used unmodified under their respective licences: the Solana test validator (Anza/Solana Labs), the Yellowstone gRPC Geyser plugin (Triton One / `rpcpool`), PostgreSQL, Prisma, Next.js, React, Tailwind CSS, shadcn/ui, Recharts and Vitest. The architecture, database schema, indexer logic, API layer, user interface, containerisation and test suite described in this report are our own contribution.

No portion of this work has been submitted in support of an application for another degree or qualification at this or any other institution.

Garvit Dadheech
B.Sc. Computer Science (2025–2026)
Date: _____

Shubh Kesharwani
B.Sc. Computer Science (2025–2026)
Date: _____

Acknowledgement

We are grateful to our project supervisor, **Dharma Teja**, for the direction, review and encouragement that shaped this work — particularly for pushing us to justify every architectural decision rather than merely describe it.

We thank the maintainers of the Solana validator client and of the Yellowstone gRPC Geyser plugin, whose open-source work made a real streaming interface available to us at zero cost; without it this project would have been reduced to polling an RPC endpoint. We also thank the Prisma, Next.js and shadcn/ui communities, whose documentation was consistently better than it had to be.

Finally, we thank the Department of Computer Science for providing the academic framework within which a systems project of this scope could be attempted.

ABSTRACT

Anyone who runs a private Solana validator — a protocol team on a local test validator, a QA engineer reproducing a bug, an infrastructure operator on a devnet fork — immediately loses the ability to inspect their own chain. Public block explorers index public clusters only; they cannot see a private ledger. The alternative, building a bespoke explorer, requires competence in gRPC streaming, Geyser plugins, relational schema design and frontend engineering, and is realistically several days of work before the first transaction becomes visible.

This project delivers **Solana Local Explorer**: a complete, self-hosted block explorer and real-time transaction indexer that a developer can bring up against a private validator with a single command, `docker compose up --build -d`. The system is a four-tier stack orchestrated by Docker Compose. A containerised `solana-test-validator` is compiled together with the Yellowstone gRPC Geyser plugin so that it emits a live stream of slot and transaction events. A TypeScript indexer subscribes to that stream, decodes each payload from its wire representation (base-58 signatures and account keys, 64-bit integers, transaction metadata), and persists it to PostgreSQL through Prisma using idempotent upserts keyed on unique constraints, so that stream replays cannot create duplicates. A Next.js 16 application then exposes the indexed data both as a set of REST endpoints and as an interactive user interface with a live dashboard, paginated transaction browsing, full transaction detail, per-account balance history with amount filtering, and prefix search across signatures and addresses.

Beyond the local validator case, the indexer accepts any Yellowstone-compatible gRPC endpoint through a single environment variable, so the same stack can be pointed at a devnet, testnet or mainnet provider without a code change. Correctness of the two most failure-prone pure-logic layers — gRPC payload parsing and `BigInt` serialisation — is protected by an automated Vitest suite of 38 unit tests, all of which pass; a further 54 documented functional, API, interface and non-functional test cases define the reproducible validation procedure for the running system. The result is a working, tested and fully documented tool that reduces observability for a private Solana network from a multi-day engineering task to a single command.

KEYWORDS

Solana · blockchain indexing · block explorer · Geyser plugin · Yellowstone gRPC · event streaming · PostgreSQL · Prisma · Next.js · Docker Compose · developer tooling · observability

TABLE OF CONTENTS

Certificate of Approval 1

Declaration of Originality & Acknowledgement 2

Abstract 3

1. Introduction 7

1.1 Background 7

1.2 Problem Statement 7

1.3 Motivation 8

1.4 Unique Value Proposition 8

1.5 Objectives 9

1.6 Scope and Delimitations 9

1.7 Organisation of the Report 10

2. Literature Review and Existing Systems 11

2.1 Public Hosted Block Explorers 11

2.2 RPC Polling Indexers 11

2.3 The Geyser Plugin Interface and Yellowstone gRPC 11

2.4 Managed Indexing and Data Platforms 12

2.5 Comparative Analysis 12

2.6 Research Gap 13

3. Requirements Analysis 14

3.1 Stakeholders and Actors 14

3.2 Use Case Model 14

3.3 Functional Requirements 16

3.4 Non-Functional Requirements 17

3.5 Assumptions and Constraints 18

4. System Design and Architecture 19

4.1 Architectural Overview 19

4.2 Key Design Decisions and Rationale 20

4.3 Data Model 21

4.4 Indexing Pipeline 23

4.5 API Design 24

4.6 User Interface Design 24

4.7 Deployment Design 25

5. Implementation 27

5.1 Technology Stack 27

5.2 Repository Layout 28

5.3 The Validator Image 28

5.4 The Indexer Service 29

5.5 Database and Migrations 31
5.6 The Explorer API Layer 31
5.7 The Explorer User Interface 33
5.8 Containerisation and Orchestration 33

6. Testing and Validation 34
6.1 Test Strategy 34
6.2 Test Environment 34
6.3 Unit Test Results 35
6.4 Integration Test Cases 37
6.5 API Test Cases 38
6.6 User Interface Test Cases 39
6.7 Non-Functional Test Cases 40
6.8 Defects Found and Resolved 40
6.9 Requirements Traceability Matrix 41
6.10 Validation Summary 42

7. Results and Analysis 43
7.1 The Delivered System 43
7.2 Objective-by-Objective Assessment 45
7.3 Performance Characteristics 46
7.4 Comparison with Alternatives 46

8. Challenges and Limitations 47

9. Conclusion and Future Work 49

References 51

Appendix A — REST API Reference 52

Appendix B — Database Schema DDL 55

Appendix C — Command Reference 57

Appendix D — Automated Test Console Output 59

List of Figures

FIGURE	TITLE	SECTION
4.1	Four-tier system architecture and container boundaries	4.1
3.1	Use case model — actors and system boundary	3.2
4.2	Entity-relationship diagram of the PostgreSQL schema	4.3
4.3	End-to-end sequence: ingestion phase and query phase	4.4
4.4	Orchestrated startup and health-gated dependency graph	4.7
7.1	Dashboard — live statistics, activity chart, recent transactions	7.1
7.2	Transaction list — paginated browsing	7.1

FIGURE	TITLE	SECTION
7.3	Transaction detail — accounts, instructions, fees, logs	7.1
7.4	Account detail — balance history with amount filtering	7.1

List of Tables

TABLE	TITLE	SECTION
2.1	Comparative analysis of existing approaches	2.5
3.1	Use case register (UC-1 … UC-10)	3.2
3.2	Functional requirements (FR-1 … FR-22)	3.3
3.3	Non-functional requirements (NFR-1 … NFR-10)	3.4
4.1	Design decisions, alternatives considered and rationale	4.2
4.2	Relational entities and their responsibilities	4.3
4.3	REST endpoint summary	4.5
5.1	Technology stack with versions and justification	5.1
6.1	Test levels, tooling and coverage	6.1
6.2	Unit test results — explorer serialisation (9 cases)	6.3
6.3	Unit test results — indexer gRPC parsing (29 cases)	6.3
6.4	Integration test cases (IT-01 … IT-12)	6.4
6.5	API test cases (API-01 … API-18)	6.5
6.6	User interface test cases (UI-01 … UI-16)	6.6
6.7	Non-functional test cases (NF-01 … NF-08)	6.7
6.8	Defect log	6.8
6.9	Requirements traceability matrix	6.9
7.1	Objective achievement summary	7.2

CHAPTER 1

Introduction

1.1 Background

Solana is a high-throughput public blockchain whose validator client is designed around a single-leader, pipelined execution model. For developers building on it, the practical unit of work is the *transaction*: a signed message that names a set of accounts, carries one or more instructions, consumes compute units, pays a fee, and either succeeds or fails with a program error. Understanding what a program did — and why it failed — almost always reduces to reading transactions and the account balance changes they produced.

On the public clusters (mainnet-beta, devnet, testnet) this is a solved problem. Hosted block explorers index every transaction and present it through a web interface. A developer pastes a signature, sees the decoded instructions, the pre- and post-balances of each account, the compute units consumed and the program logs, and moves on.

The situation changes completely the moment the chain in question is *private*. Solana ships a tool called `solana-test-validator` that runs a complete single-node cluster on a developer's own machine. It is the standard environment for program development: it starts in seconds, produces deterministic ledgers, allows unlimited airdrops, and can be reset at will. It is also, by construction, invisible to the outside world. No hosted explorer can index a ledger that exists only inside one laptop. The same is true of a private devnet fork, a CI cluster spun up for an integration test, or a validator behind a corporate firewall.

Solana anticipates this class of problem through the **Geyser plugin interface**: a dynamic-library hook inside the validator that receives account updates, slot status changes, transaction notifications and block metadata as they are produced, and can forward them anywhere. The most widely deployed implementation of that hook is **Yellowstone gRPC**, maintained by Triton One, which exposes the validator's internal event firehose as a gRPC streaming service. Yellowstone is the same mechanism that commercial Solana RPC providers use behind the scenes to power their own indexing products.

What Yellowstone provides, however, is a *stream* — not a database, not an API and certainly not a user interface. Turning that stream into something a developer can actually look at is the gap this project addresses.

1.2 Problem Statement

> **PROBLEM**
> A developer running a private Solana validator has no practical way to inspect the transactions their own chain is producing. Public explorers cannot see private ledgers, and building a purpose-made explorer requires simultaneous competence in gRPC streaming, Geyser plugin deployment, relational schema design, ORM tooling and front-end engineering — a multi-day undertaking that has to be repeated by every team that hits the same wall.

The problem decomposes into four concrete obstacles, each of which independently blocks a developer who tries to solve it themselves:

1. **The data is not exposed by default.** A stock `solana-test-validator` emits no event stream. Enabling one means compiling the Yellowstone Geyser plugin from Rust source against a version of the plugin that exactly matches the validator's ABI, writing a plugin configuration file, and starting the validator with the

right flags. Version mismatches produce dynamic-link failures at validator start-up rather than clear error messages.

2. **The stream is not queryable.** gRPC delivers ephemeral, push-based messages. Answering "what happened five minutes ago?" or "what is this account's balance history?" requires the consumer to durably store and index every message. Nothing about the stream itself provides pagination, aggregation or search.
3. **The wire format is not directly usable.** Signatures and account keys arrive as raw byte buffers that must be base-58 encoded; balances, fees, slots and compute units are 64-bit integers that exceed the range of an IEEE-754 double and therefore cannot survive an ordinary `JSON.stringify` round trip; instruction data is opaque and must be encoded before storage. Each of these is a small problem, and each of them silently corrupts data if handled naively.
4. **The setup is not reproducible.** Even after a working prototype exists, it typically depends on locally installed toolchains — a specific Rust version, a specific Solana CLI release, a local PostgreSQL instance, a Node runtime — so it works on the author's machine and nowhere else. A teammate or an examiner cannot run it.

1.3 Motivation

The motivation for the project is that this gap is both *common* and *disproportionately expensive*. It is common because every Solana program team develops against a local validator; it is expensive because the workarounds developers actually adopt are poor:

- **Reading validator logs.** The validator writes program logs to stdout. This works for a single transaction under a debugger and collapses immediately for anything involving history, aggregation, or more than a handful of transactions.
- **Ad-hoc RPC scripts.** Calling `getSignaturesForAddress` followed by `getTransaction` in a loop produces a one-off answer to a single question, has to be rewritten for the next question, and adds polling load to the validator.
- **Deploying to devnet to "see it in an explorer".** This is the most common workaround and the most damaging one: it converts a fast local feedback loop into a slow remote one, exposes work-in-progress programs publicly, and consumes airdropped devnet SOL.

A tool that removes this friction pays for itself the first time it is used. That observation — not the novelty of any individual component — is what motivated the project.

1.4 Unique Value Proposition

UNIQUE VALUE PROPOSITION

Solana Local Explorer is the only self-hosted tool that turns a private Solana validator into a fully browsable, queryable block explorer with a single command — no gRPC knowledge, no database setup, no frontend work, and no need to publish anything to a public cluster.

The proposition rests on four differentiators, each of which is independently verifiable against the delivered system:

DIFFERENTIATOR	WHAT IT MEANS IN PRACTICE
One command, zero prerequisites	`docker compose up --build -d` builds and starts the validator, the Geyser plugin, PostgreSQL, the indexer and the web UI, in the correct order, gated on health checks. The host needs only Docker — no Rust toolchain, no Solana CLI, no Node runtime, no PostgreSQL installation.
Works where public explorers structurally cannot	The system indexes the ledger it is attached to, whatever that ledger is. A local test validator, a private fork, a CI cluster and an air-gapped network are all first-class, because nothing in the pipeline depends on the chain being publicly reachable.
Push-based, not poll-based	Data arrives over the validator's own Geyser stream rather than through repeated RPC polling. Transactions appear in the UI within the dashboard's five-second refresh interval, and the validator is never asked a question it did not volunteer to answer.
Not locked to local	Setting `YELLOWSTONE_ADDR` (and optionally `YELLOWSTONE_XTOKEN`) re-points the same indexer at any Yellowstone-compatible endpoint, including commercial devnet, testnet and mainnet providers. The local validator is the default, not a limitation.

Section 2.5 positions this proposition against the existing alternatives, and Section 7.4 returns to it with the delivered system in hand.

1.5 Objectives

The project set out to achieve six objectives, stated here in the form used throughout the report.

ID	OBJECTIVE
OBJ–1	Produce a containerised Solana test validator that runs the Yellowstone gRPC Geyser plugin, compiled from source against a matching validator version, and exposes both the JSON-RPC endpoint and the gRPC stream.
OBJ–2	Implement a resilient indexer service that subscribes to the gRPC stream, correctly decodes every field of the transaction payload, and persists it to a relational database without creating duplicates on replay or restart.
OBJ–3	Design a normalised PostgreSQL schema, with migrations, that records transactions, failed transactions, accounts, account–transaction associations and per-transaction account balance changes, and that is indexed for the queries the interface actually issues.
OBJ–4	Expose the indexed data through a documented REST API and an interactive web interface providing a live dashboard, paginated transaction browsing, transaction detail, account balance history with amount filtering, and prefix search.
OBJ–5	Orchestrate the entire stack with Docker Compose so that a first-time user can start it with one command, with correct dependency ordering and health gating, and reset it cleanly.
OBJ–6	Validate the system with an automated unit test suite over the parsing and serialisation logic, plus a documented, reproducible functional test plan covering the API, the interface and the non-functional requirements.

1.6 Scope and Delimitations

IN SCOPE

- Streaming ingestion of slot and transaction events from a Yellowstone gRPC endpoint.
- Durable, idempotent storage of successful and failed transactions.

OUT OF SCOPE

- **Instruction decoding by IDL.** Instructions are stored in raw (program-index, accounts, base-58 data) form; they are not decoded into named

- Per-transaction pre-balance, post-balance and net change for every account key.
- Account registry with first-seen and last-seen timestamps.
- REST API with pagination, filtering and prefix search.
- Web interface: dashboard, transaction list and detail, account detail, global search.
- Single-command Docker Compose deployment and reset.
- Automated unit tests and a documented functional test plan.

program methods, which would require per-program IDLs.

- **SPL token and NFT semantics.** Balances are tracked in lamports only; token account balances and metadata are not interpreted.
- **Multi-validator or clustered deployment.** The stack targets a single validator and a single indexer instance.
- **Authentication and multi-tenancy.** The explorer is a local developer tool and assumes a trusted network.
- **Historical backfill.** Indexing begins when the indexer connects; transactions produced before that point are not retrieved.
- **Write operations.** The explorer is strictly read-only with respect to the chain; it never submits transactions.

1.7 Organisation of the Report

Chapter 2 surveys the existing approaches to Solana data access — hosted explorers, RPC polling, the Geyser interface and managed indexing platforms — and identifies the gap the project fills. **Chapter 3** derives the requirements: actors, use cases, and numbered functional and non-functional requirements that the rest of the report traces against. **Chapter 4** presents the architecture, the design decisions and the alternatives rejected, the data model, the ingestion pipeline, the API contract and the deployment topology. **Chapter 5** describes the implementation of each component with reference to the actual source. **Chapter 6** presents the test strategy, the full test case catalogue, the executed automated results, the defect log and the traceability matrix. **Chapter 7** evaluates the delivered system against the objectives. **Chapter 8** states the challenges encountered and the limitations that remain, and **Chapter 9** concludes and sets out future work. Four appendices provide the API reference, the schema DDL, a command reference and the verbatim test console output.

CHAPTER 2

Literature Review and Existing Systems

This chapter surveys the four families of approach that a developer might reach for when they need to inspect Solana transaction data, evaluates each against the requirements of a private validator, and states the resulting gap.

2.1 Public Hosted Block Explorers

The dominant tools in the ecosystem are hosted explorers: the official Solana Explorer, Solscan, SolanaFM and XRAY, among others. Architecturally they are all variations on the same design — a fleet of validators or RPC nodes feeds a proprietary indexing pipeline, which populates a search-optimised datastore behind a web front end. They provide decoded instructions, token transfer summaries, program labels, address books and historical charts, and they do so at a level of polish that an individual project cannot match.

Their limitation is structural rather than technical. A hosted explorer can only display a ledger that it indexes, and it indexes only the public clusters. There is no mechanism by which `solana-test-validator` running on a laptop can be made visible to Solscan, because the laptop's ledger is not part of any cluster the explorer participates in. The problem is not that these tools are missing a feature; it is that the data never reaches them.

A secondary limitation matters for teams working on unreleased protocols: publishing to devnet purely to obtain explorer visibility makes work-in-progress programs and their transaction patterns publicly observable.

2.2 RPC Polling Indexers

The classic self-hosted approach is to poll the validator's JSON-RPC interface. A consumer calls `getSignaturesForAddress` to discover new signatures for an address of interest, then `getTransaction` for each signature, and writes the result somewhere. This is the mechanism behind most tutorial-grade Solana indexers and behind a large amount of ad-hoc script code in the ecosystem.

It works, and it has three well-known weaknesses:

- **It is address-scoped, not chain-scoped.** `getSignaturesForAddress` requires an address. Discovering *all* transactions on a chain — which is what an explorer needs — means either knowing every address in advance or walking blocks with `getBlock`, which is substantially heavier.
- **Latency is bounded below by the polling interval.** A one-second poll produces up to one second of lag and issues 3,600 requests per hour whether or not anything happened. Shortening the interval trades validator load for freshness, and there is no setting that avoids the trade-off.
- **It creates load on the node being observed.** On a local test validator this is tolerable; against a shared or rate-limited RPC endpoint it is the first thing to break.

Polling is therefore adequate for narrow, address-specific monitoring and unsuitable as the foundation of a general explorer.

2.3 The Geyser Plugin Interface and Yellowstone gRPC

Solana's answer to the polling problem is the **Geyser plugin interface**. The validator loads a shared library at start-up (configured through a JSON plugin file) and invokes callbacks on it as events occur: account updates, slot status transitions, transaction notifications, block metadata and entry notifications. Because the plugin

runs inside the validator process, it observes events at the moment they are produced, with no polling and no RPC round trip.

The Geyser interface is a Rust ABI, which means writing a plugin directly requires Rust and, more awkwardly, requires the plugin to be compiled against the exact validator version it will be loaded into. **Yellowstone gRPC** (Triton One, `rpcpool/yellowstone-grpc`) solves this once for everyone: it is a production-grade Geyser plugin that re-exposes the entire event firehose as a gRPC streaming service defined by a Protobuf schema. A consumer opens a bidirectional stream, sends a `SubscribeRequest` describing which event classes and filters it wants, and then receives `SubscribeUpdate` messages until it disconnects.

Yellowstone is the interface that commercial Solana infrastructure providers expose to customers under names such as "gRPC streaming" or "geyser endpoints", and it is the interface this project builds on. It is, however, deliberately a transport: it stores nothing, indexes nothing, and answers no questions. Everything downstream — durability, schema, query, presentation — is left to the consumer.

2.4 Managed Indexing and Data Platforms

A fourth family of solutions removes the infrastructure burden entirely by hosting it. Webhook and enhanced-API providers (for example Helius) push decoded, enriched transaction events to a customer endpoint. Query platforms (Dune, Flipside, Bitquery) expose curated Solana datasets through SQL or GraphQL. Framework-based indexers (The Graph's Substreams, SubQuery) let a developer define transformation logic that a hosted network executes.

These platforms are the right answer for production analytics on public clusters, and they are the wrong answer for this problem for three reasons. First, and decisively, they index public chains; a private validator's data never reaches them, for exactly the same reason it never reaches Solscan. Second, they introduce an external dependency — an account, an API key, a rate limit and, in most cases, a bill — into what should be a local development loop. Third, sending private pre-release transaction data to a third-party service is a disclosure that many teams cannot make.

2.5 Comparative Analysis

Table 2.1 evaluates the four approaches, plus this project, against the criteria that matter for a private validator.

APPROACH	WORKS ON A PRIVATE LEDGER	PUSH-BASED (NO POLLING)	DURABLE + QUERYABLE	SELF-HOSTED / NO THIRD PARTY	READY-MADE UI	SETUP EFFORT
Hosted public explorers Solana Explorer, Solscan, SolanaFM	✗	n/a	✓	✗	✓	None
RPC polling scripts getSignaturesForAddress + getTransaction	✓	✗	Partial	✓	✗	Low, per query
Raw Geyser / Yellowstone gRPC stream only	✓	✓	✗	✓	✗	High

APPROACH	WORKS ON A PRIVATE LEDGER	PUSH-BASED (NO POLLING)	DURABLE + QUERYABLE	SELF-HOSTED / NO THIRD PARTY	READY-MADE UI	SETUP EFFORT
Managed indexing platforms Helius, Dune, Bitquery, Substreams	✗	✓	✓	✗	Partial	Low
Solana Local Explorer this project	✓	✓	✓	✓	✓	**One command**

Table 2.1 — Comparative analysis of existing approaches against the requirements of a private Solana validator.

2.6 Research Gap

The survey shows that the individual pieces required to solve this problem all exist and are mature: the Geyser interface exposes the events, Yellowstone transports them, PostgreSQL can store them, and modern web frameworks can present them. What does not exist is an **integrated, self-hosted, one-command assembly of those pieces aimed at a private validator**.

Every row of Table 2.1 fails on at least one criterion that a private-validator workflow makes mandatory. Hosted explorers and managed platforms fail on the first criterion, which is fatal and cannot be engineered around. Polling fails on latency and load. Raw Yellowstone fails on everything downstream of transport — it is a starting point that leaves the hard parts (durability, schema, query, interface, deployment) to be re-solved by each team independently.

> **GAP ADDRESSED BY THIS PROJECT**
>
> The contribution of this work is *integration*, not invention: it composes a validator, a Geyser plugin, a streaming consumer, a normalised relational schema, a REST layer and a web interface into a single reproducible artefact, so that the multi-day engineering task identified in Section 1.2 collapses to one command.

CHAPTER 3

Requirements Analysis

3.1 Stakeholders and Actors

ACTOR	TYPE	GOALS AND EXPECTATIONS
dApp / Protocol Developer	Primary, human	Wants to see what their program did on the local validator: which accounts a transaction touched, how much it cost, whether it failed and why. Values speed of the feedback loop above completeness of features.
QA / Test Engineer	Primary, human	Wants to reproduce and evidence a defect: locate a specific transaction, capture the instruction set and program logs, and trace balance changes across a sequence of transactions.
Infrastructure Operator	Secondary, human	Wants the stack to start, stay up and reset predictably; wants to re-point it at a non-local gRPC endpoint without editing code; wants clear signals when a component is unhealthy.
Yellowstone gRPC source	Supporting, system	Supplies the event stream. Either the bundled local validator or an external devnet/testnet/mainnet provider. Expects a well-formed `SubscribeRequest` and a consumer that keeps up with the stream.
Automated consumer	Secondary, system	Any script, dashboard or test harness that reads the REST API instead of the UI. Expects stable JSON shapes and predictable pagination.

3.2 Use Case Model

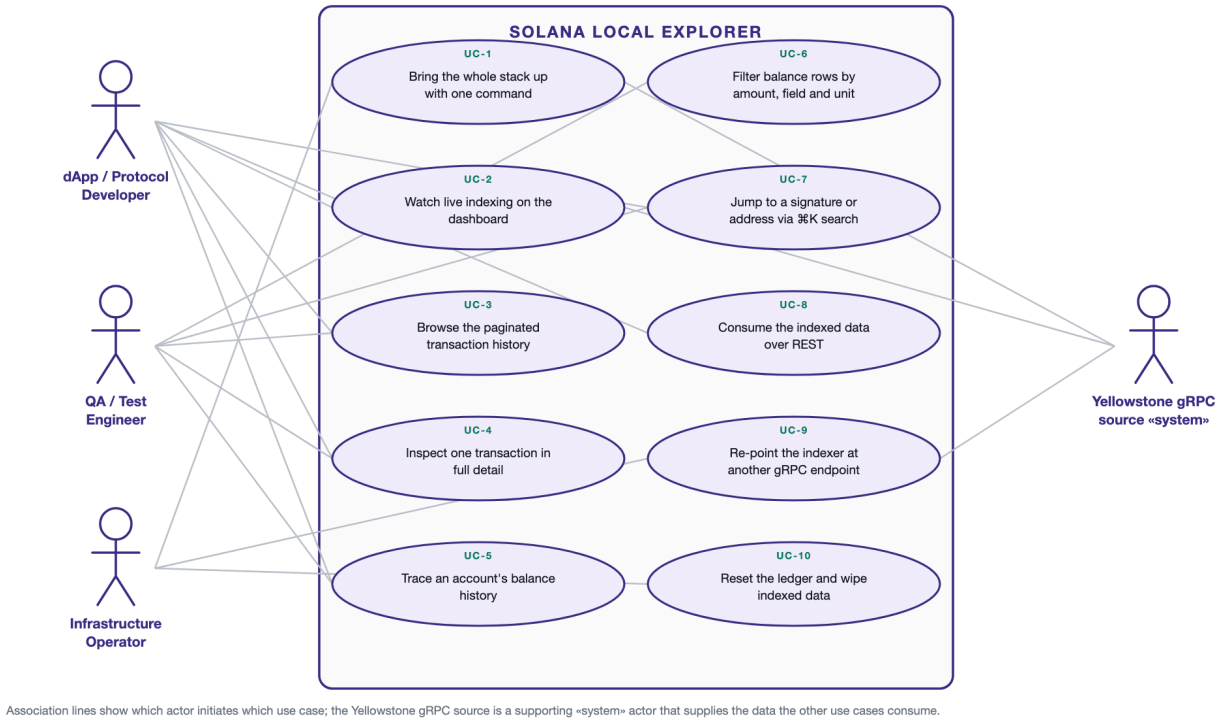


Figure 3.1 — Use case model showing the three human actors, the supporting gRPC system actor, and the ten use cases inside the system boundary.

ID	USE CASE	PRIMARY ACTOR	MAIN SUCCESS SCENARIO (ABBREVIATED)
UC-1	Bring the whole stack up with one command	Infrastructure Operator	Operator runs `docker compose up --build -d`; images build; validator and database start and become healthy; indexer migrates the schema and subscribes; explorer serves on :3000.
UC-2	Watch live indexing on the dashboard	Developer	User opens `/`; sees total transactions, failed count, success rate and latest slot; the activity chart and recent-transaction table refresh every five seconds without a page reload.
UC-3	Browse the paginated transaction history	Developer, QA	User opens `/transactions`; sees a page of transactions ordered by descending slot; navigates pages; page state is reflected in the URL.
UC-4	Inspect one transaction in full detail	Developer, QA	User selects a signature; sees status, slot, fee, compute units, the ordered account list with links, the instruction set, memos and, for failures, the error and program logs.
UC-5	Trace an account's balance history	QA	User opens an account page; sees address, first/last seen, latest balance, and a paginated table of pre-balance, post-balance and net change per transaction.
UC-6	Filter balance rows by amount, field and unit	Developer, QA	User chooses a field (post/pre/change), a unit (SOL or lamports) and an exact value or a min/max range; the table and its pagination reflect the filtered result set.
UC-7	Jump to a signature or address via ⌘K search	Developer	User presses ⌘K, types at least eight characters; matching signatures and addresses are offered; selecting one navigates to the corresponding detail page.
UC-8	Consume the indexed data over REST	Automated consumer	Client issues `GET` to one of the five endpoints and receives JSON with 64-bit values encoded as strings and a consistent pagination envelope.
UC-9	Re-point the indexer at another gRPC endpoint	Infrastructure Operator	Operator sets `YELLOWSTONE_ADDR` (and optionally an x-token) and restarts; the indexer connects to the external endpoint and indexes that chain instead.
UC-10	Reset the ledger and wipe indexed data	Infrastructure Operator	Operator runs `docker compose down -v`; named volumes are removed; the next start produces a fresh ledger and an empty, freshly migrated database.

Table 3.1 — Use case register.

3.2.1 Expanded use case: UC-4, Inspect one transaction in full detail

Precondition	The transaction has been indexed; the user knows or can find its signature.
Trigger	The user selects a signature from the dashboard, the transaction list, an account's balance history, or the ⌘K search.
Main flow	1. The client requests `GET /api/transactions/{signature}`. 2. The API looks the signature up in `Transaction`, including related memos. 3. If found, all 64-bit fields are serialised to strings and returned. 4. The page renders status, slot, fee in SOL, compute units, the ordered account list (each linked to its account page), the instruction list and any memos.
Alternate flow A	The signature is absent from `Transaction` but present in `FailedTransaction`: the record is returned with `success: false` and the page additionally renders the error string and the captured program logs.
Exception flow	The signature is in neither table: the API returns `404` with an error body and the page shows a "Transaction not found" state with a route back to the transaction list.
Postcondition	No state is modified; the operation is strictly read-only.

3.2.2 Expanded use case: UC-6, Filter balance rows by amount

Precondition	The account exists in the `Account` table and has at least one balance row.
Main flow	1. The user selects the field to search (`postBalance`, `preBalance` or `balanceChange`) and the unit (SOL or lamports). 2. The user enters either an exact amount or a minimum and/or maximum. 3. On submit, the values become query parameters on the account URL, so the filtered view is linkable and survives a reload. 4. The API converts SOL values to lamport `BigInts`, builds the corresponding `equals` / `gte` / `lte` predicate, and applies it to the balance query, the count and the aggregate. 5. The table, the total and the pagination all reflect the filtered set; the latest-balance figure continues to reflect the unfiltered account state.
Business rules	An exact `amount` takes precedence over `minAmount`/`maxAmount`. An unrecognised field name falls back to `postBalance`. A non-finite or empty amount is ignored rather than treated as zero.

3.3 Functional Requirements

ID	AREA	REQUIREMENT	PRIORITY
FR–1	Ingestion	The system shall open a gRPC subscription to a Yellowstone endpoint for both slot updates and all transactions, at `PROCESSED` commitment.	Must
FR–2	Ingestion	The system shall accept the endpoint as `host:port` or as a full `http(s)://` URL, normalising the former to the latter.	Must
FR–3	Ingestion	The system shall support an optional authentication token for endpoints that require one.	Should
FR–4	Parsing	The system shall base-58 encode transaction signatures and account public keys received as byte buffers, preserving account key order.	Must
FR–5	Parsing	The system shall extract each instruction's program index, account index list and base-58 encoded data, together with its ordinal position.	Must
FR–6	Parsing	The system shall extract fee and compute units consumed as 64-bit integers, and classify a transaction as failed when transaction metadata carries a non-null error.	Must
FR–7	Persistence	The system shall persist every successful transaction with signature, slot, block time, fee, compute units, account list and instruction list.	Must
FR–8	Persistence	The system shall persist every failed transaction with signature, slot, serialised error and captured program logs.	Must
FR–9	Persistence	The system shall record, for each account key index of a transaction, the pre-balance, post-balance and computed net change.	Must
FR–10	Persistence	The system shall maintain an account registry with first-seen and last-seen timestamps and a many-to-many association to transactions.	Must
FR–11	Persistence	All writes shall be idempotent: re-processing the same transaction shall update rather than duplicate, enforced by unique constraints.	Must
FR–12	Persistence	The system shall apply pending database migrations automatically at indexer start-up, retrying while the database is still initialising.	Must
FR–13	API	The system shall expose aggregate statistics: total transactions, total failures, success rate and latest indexed slot, with the ten most recent transactions.	Must
FR–14	API	The system shall expose a paginated transaction list ordered by descending slot, with a bounded page size.	Must
FR–15	API	The system shall expose a single transaction by signature, falling back to the failed-transaction table before returning 404.	Must

ID	AREA	REQUIREMENT	PRIORITY
FR–16	API	The system shall expose account detail with latest balance, aggregate net change, and a paginated balance history including the linked transaction signature.	Must
FR–17	API	The system shall support filtering balance history by field, unit and exact or ranged amount.	Should
FR–18	API	The system shall expose prefix search across transaction signatures and account addresses, requiring a minimum query length.	Should
FR–19	API	All API responses shall encode 64-bit integers as strings and timestamps as ISO-8601, so that no precision is lost in JSON.	Must
FR–20	UI	The interface shall provide a dashboard, a transaction list, a transaction detail page, an account detail page and a global search dialog, with cross-links between them.	Must
FR–21	UI	The dashboard shall refresh automatically at a fixed interval without a full page reload, and shall render a time-bucketed success/failure activity chart with selectable ranges.	Should
FR–22	Deployment	The whole stack shall start from a single command with correct dependency ordering, and shall be resettable to a clean state.	Must

Table 3.2 — Functional requirements.

3.4 Non-Functional Requirements

ID	QUALITY ATTRIBUTE	REQUIREMENT AND ACCEPTANCE CRITERION
NFR–1	Usability	A first-time user with only Docker installed shall reach a working explorer using one documented command. Acceptance: the Quick Start in the README is sufficient, with no undocumented prerequisite.
NFR–2	Latency	A transaction shall become visible in the dashboard within one refresh cycle (five seconds) of being produced, under normal local load.
NFR–3	Reliability	The indexer and explorer shall restart automatically on failure (`restart: unless-stopped`), and the indexer shall tolerate a database that is not yet accepting connections.
NFR–4	Data integrity	No transaction shall be stored twice and no 64-bit value shall lose precision anywhere in the pipeline. Acceptance: unique constraints in the schema and an automated serialisation test suite.
NFR–5	Performance	List and detail queries shall be served by database indexes rather than sequential scans; page size shall be bounded to prevent unbounded result sets.
NFR–6	Portability	The stack shall run on macOS (including Apple Silicon, via `linux/amd64` emulation), Linux and Windows with WSL2, without source changes.
NFR–7	Maintainability	The codebase shall be typed end to end (TypeScript with a generated Prisma client) and the schema shall evolve only through checked-in migrations.
NFR–8	Configurability	The data source and database connection shall be configurable through environment variables alone, with sensible defaults.
NFR–9	Observability	Each indexed transaction and each connection state change shall be logged, so that `docker compose logs indexer` is sufficient to diagnose ingestion.
NFR–10	Testability	Parsing and serialisation logic shall be exercisable without a running validator or database, so that the automated suite runs in seconds in any environment.

Table 3.3 — Non-functional requirements.

3.5 Assumptions and Constraints

ASSUMPTIONS

- The host runs Docker Desktop 24.x or newer with Compose V2 and has roughly 4 GB of free RAM and 4 GB of free disk.
- Ports 3000, 8899, 10000 and 5433 are available on the host, or are remapped by the operator.
- The network is trusted: the explorer performs no authentication and is intended for localhost or a private network.
- Outbound internet access is available for the first build only, to fetch base images and the Geyser plugin source.

CONSTRAINTS

- The Geyser plugin must be built against a validator version with a compatible ABI; the project pins validator 1.18.26 and plugin 1.15.3.
- The validator image is `linux/amd64`, so Apple Silicon hosts run it under emulation and pay a first-build penalty.
- Indexing is forward-only from the moment of connection; there is no historical backfill.
- The Solana wire format supplies raw instruction data only; decoding it into named program methods would require per-program IDLs, which are out of scope.

CHAPTER 4

System Design and Architecture

4.1 Architectural Overview

The system is a **four-tier, event-driven architecture** in which each tier is a separate container and communicates with its neighbours over exactly one well-defined protocol. Reading Figure 4.1 from the bottom: the blockchain tier produces events, the ingestion tier consumes and normalises them, the data tier stores them, and the presentation tier queries and displays them. Data flows strictly upward; no tier reaches past its immediate neighbour.

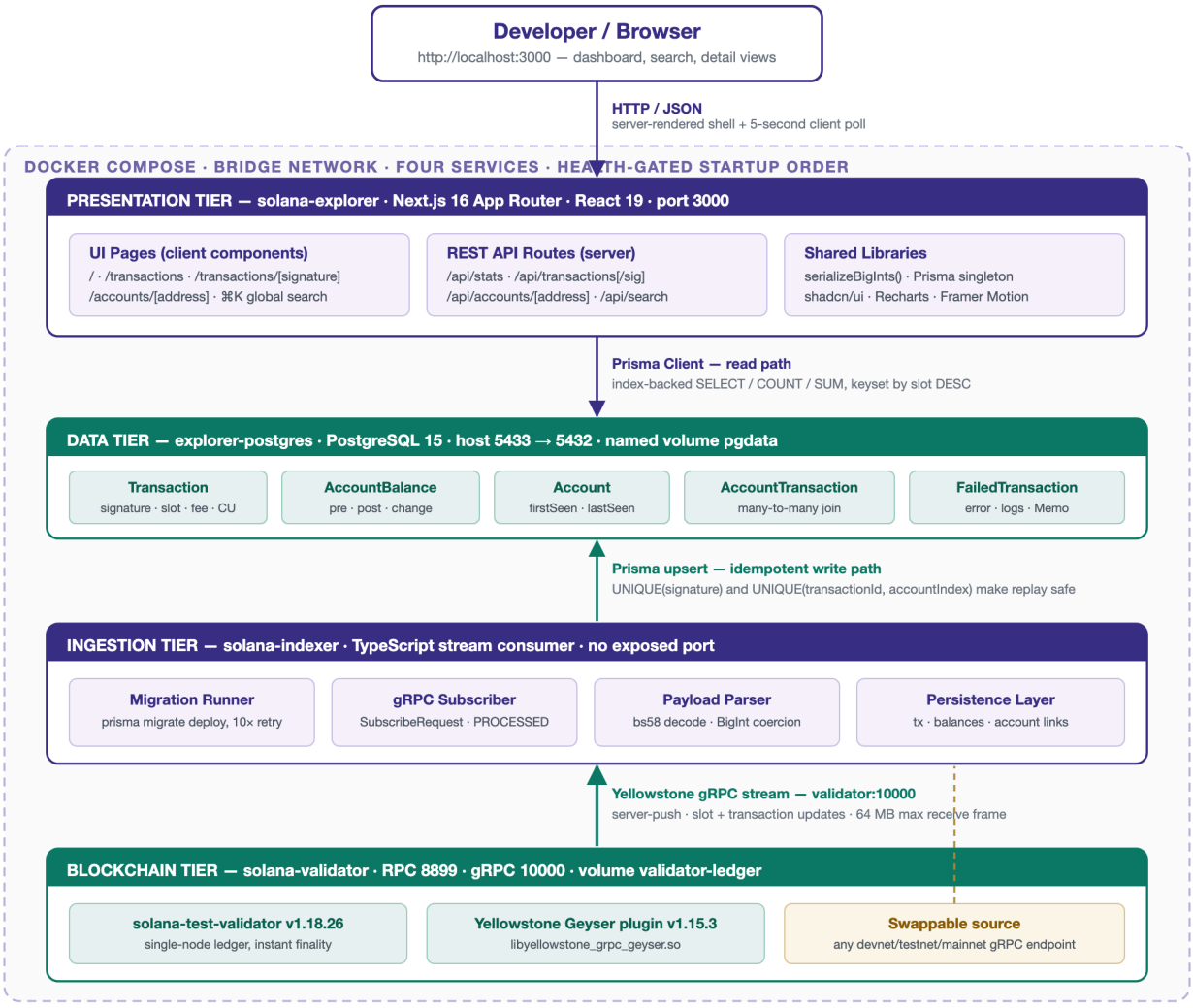


Figure 4.1 — Four-tier architecture. The dashed boundary is the Docker Compose bridge network; the browser is the only participant outside it.

TIER	CONTAINER	RESPONSIBILITY	INTERFACE OFFERED
Blockchain	`solana-validator`	Runs a single-node Solana cluster and emits every slot and transaction event through the loaded Geyser plugin.	JSON-RPC :8899 Yellowstone gRPC :10000
Ingestion	`solana-indexer`	Maintains the gRPC subscription, decodes payloads, and performs idempotent writes. Also owns schema migration.	None — outbound only

TIER	CONTAINER	RESPONSIBILITY	INTERFACE OFFERED
Data	`explorer-postgres`	Durable storage and the indexes that make list, detail, aggregate and prefix-search queries cheap.	PostgreSQL wire protocol :5432
Presentation	`solana-explorer`	Serves the user interface and the REST API; converts database representations into JSON-safe shapes.	HTTP :3000

The separation buys three properties that matter directly to the requirements:

- **Independent failure.** If the indexer crashes, the explorer continues to serve everything already indexed; if the explorer is stopped, ingestion continues uninterrupted. Only the validator is a true single point of failure, and that is inherent to the problem.
- **A substitutable data source.** Because the indexer's only inbound dependency is a gRPC address, swapping the local validator for a remote provider is a configuration change (FR-2, UC-9), not a refactor.
- **Independent scaling of read and write paths.** The write path is a single long-lived stream consumer; the read path is a stateless HTTP server. They contend only at the database, which is where contention is cheapest to reason about.

4.2 Key Design Decisions and Rationale

Table 4.1 records the decisions that shaped the system, the alternatives that were considered, and why each was resolved as it was. These are the questions most likely to be raised in a design review.

DECISION	ALTERNATIVES CONSIDERED	RATIONALE
D-1 Consume Geyser via Yellowstone gRPC	Poll JSON-RPC; write a bespoke Geyser plugin in Rust	Polling cannot see all transactions without walking blocks and adds latency proportional to the interval (Section 2.2). A bespoke plugin would require Rust and would have to be recompiled for every validator release. Yellowstone gives push semantics with a stable Protobuf contract and a maintained TypeScript client.
D-2 Subscribe at `PROCESSED` commitment	`CONFIRMED`; `FINALIZED`	The primary use case is a local test validator where the developer wants a transaction to appear as soon as it is executed. `PROCESSED` minimises latency; because writes are idempotent upserts (D-4), any later correction of the same signature overwrites rather than duplicates.
D-3 PostgreSQL rather than an embedded store	SQLite; a document store; an in-memory cache	The workload needs 64-bit integers without precision loss, native array and JSONB columns for account lists and instructions, real secondary indexes, and server-side aggregation. PostgreSQL provides all four, and running it as a container costs the user nothing.
D-4 Idempotent upserts keyed on unique constraints	Plain inserts with duplicate-key error handling; read-then-write	A gRPC stream can redeliver, and the indexer can restart mid-stream. Upserting on `signature` and on `(transactionId, accountIndex)` makes reprocessing harmless by construction rather than by defensive coding, which directly satisfies FR-11 and NFR-4.
D-5 Separate `FailedTransaction` table	A single table with a nullable error column	Failed transactions carry fields that successful ones do not (a serialised error, a full program-log array) and lack fields that successful ones do. Separating them keeps both row shapes dense and lets the dashboard compute a success rate with two cheap counts.

DECISION	ALTERNATIVES CONSIDERED	RATIONALE
D-6 Store balances per account *index*, not per address	One aggregated balance row per address per transaction	Solana's `preBalances`/`postBalances` arrays are positional: element `i` corresponds to account key `i`. Preserving the index keeps the mapping faithful to the wire format and gives a natural, collision-free uniqueness key.
D-7 `AccountBalance.accountAddress` is a plain string, not a foreign key	A foreign key to `Account.id`	Balance rows can then be written in one pass with the transaction, without first resolving every address to an account row. The cost is that an address may have balance rows before it has an `Account` row; the account upsert runs immediately afterwards, so the window is momentary. This is a deliberate trade of strict referential integrity for write throughput and simplicity.
D-8 Serialise all 64-bit values as strings in JSON	Return numbers and accept precision loss; use a custom JSON serialiser globally	`JSON.stringify` throws on `BigInt`, and coercing to `Number` silently corrupts values above 2^{53} — a real risk for lamport balances. A single recursive `serializeBigInts()` helper applied at every response boundary is small enough to unit-test exhaustively (FR-19, NFR-4).
D-9 Next.js API routes rather than a separate backend	A standalone Express or Fastify service	The API surface is five read-only endpoints over the same Prisma client the pages need. Collapsing them into one deployable removes a container, a port, a CORS configuration and a second dependency tree, at no expressive cost.
D-10 Migrations run inside the indexer at start-up	A dedicated migration container or job; manual migration	The indexer is the only writer and therefore the natural schema owner. Running `prisma migrate deploy` with a bounded retry loop makes the single-command start-up (FR-22) work even though Compose health checks cannot express "database has finished initialising its data directory".
D-11 Health-gated start-up ordering	Plain `depends_on`; sleep-and-hope in an entrypoint script	Plain `depends_on` only waits for container start, not readiness. `condition: service_healthy` plus real health checks makes the ordering deterministic, which is what makes the one-command promise reliable rather than usually true.
D-12 Prefix search with an eight-character minimum	Full-text search; trigram similarity; unbounded substring search	Signatures and addresses are base-58 identifiers, so users search by prefix, which an index can serve. A minimum length keeps a two-character query from matching a large fraction of the table, and results are capped at three per type.

Table 4.1 — Design decisions, alternatives considered and rationale.

4.3 Data Model

The schema is defined once in Prisma and compiled to SQL migrations. It comprises six entities: two transaction tables (successful and failed), an account registry, a many-to-many association, a per-index balance table and a memo table.

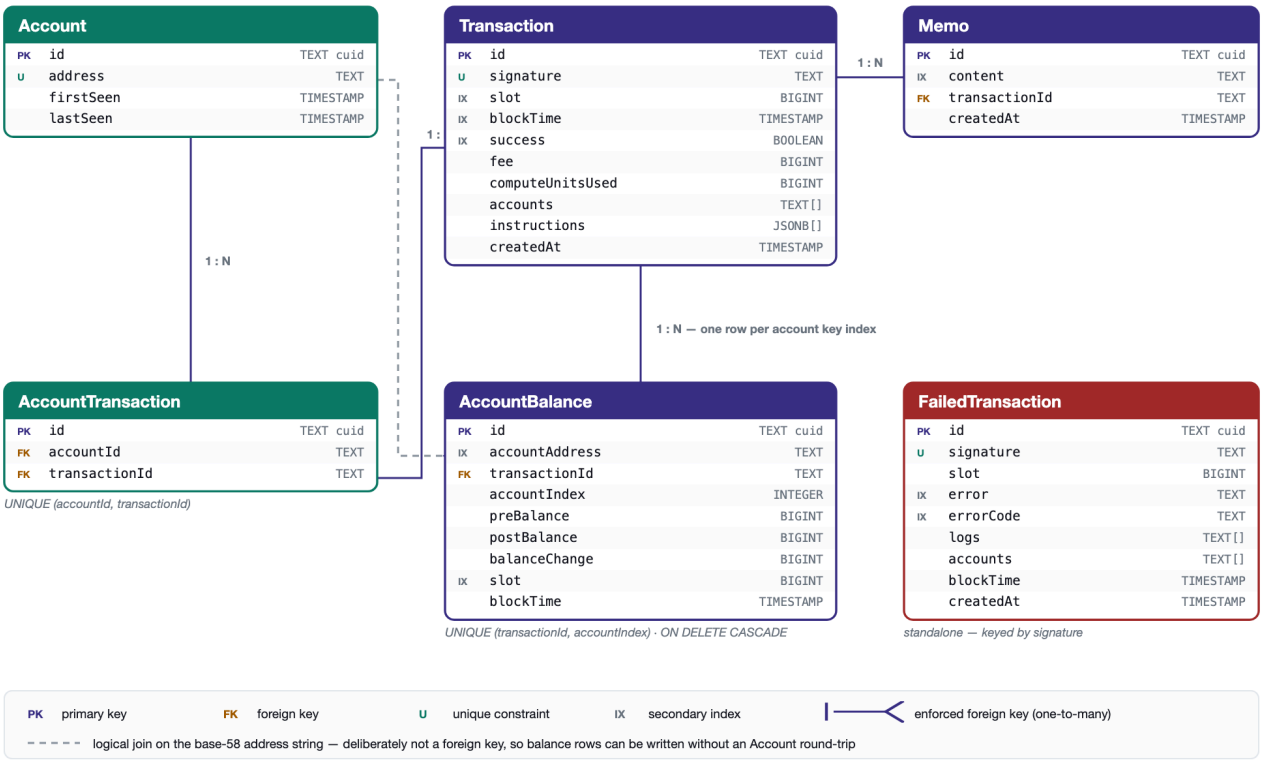


Figure 4.2 — Entity-relationship diagram. Solid connectors are enforced foreign keys; the dashed connector is a logical join on the base-58 address string (decision D-7).

ENTITY	RESPONSIBILITY AND NOTABLE DESIGN POINTS
`Transaction`	One row per successful transaction. `signature` is unique and is the natural key used for upserts. `accounts` is a `TEXT[]` preserving wire order, and `instructions` is a `JSONB[]` — both are denormalised deliberately, because the detail page always reads them together with the row and never queries into them. Secondary indexes on `slot`, `success` and `blockTime` serve the list ordering and the dashboard.
`FailedTransaction`	One row per failed transaction, holding the serialised error and the full program log array. Standalone by design (D-5); joined to nothing, looked up by signature.
`Account`	Registry of every address ever seen. `firstSeen` defaults on insert and `lastSeen` is maintained by Prisma's `@updatedAt`, so an empty update still refreshes the timestamp.
`AccountTransaction`	Join table resolving the many-to-many between accounts and transactions, with a composite unique constraint so repeated processing cannot create duplicate links.
`AccountBalance`	One row per account key index per transaction, carrying `preBalance`, `postBalance` and the precomputed `balanceChange`. Storing the difference rather than computing it on read makes the amount filter (FR-17) a plain indexed comparison. Unique on `(transactionId, accountIndex)`; cascades when its transaction is deleted.
`Memo`	Optional memo content attached to a transaction, indexed on content for lookup.

Table 4.2 — Relational entities and their responsibilities.

4.3.1 Indexing strategy

Every index in the schema exists to serve a query the interface actually issues; none were added speculatively.

INDEX	QUERY IT SERVES
`Transaction(signature) UNIQUE`	Upsert key on the write path; detail lookup on the read path.

INDEX	QUERY IT SERVES
`Transaction(slot)`	`ORDER BY slot DESC` for the list and the dashboard's latest-slot query.
`Transaction(success), (blockTime)`	Status filtering and time-bucketed activity chart data.
`AccountBalance(accountAddress)`	All balance rows for one account.
`AccountBalance(accountAddress, slot)`	Composite index serving “latest balance” — the single most frequent account query.
`AccountBalance(transactionId, accountIndex) UNIQUE`	Upsert key for balance rows.
`Account(address) UNIQUE`	Account upsert and prefix search.
`FailedTransaction(signature) UNIQUE, (error), (errorCode)`	Failed-transaction lookup and error-class grouping.

4.4 Indexing Pipeline

Figure 4.3 shows the two phases of the system's life. The ingestion phase runs continuously and is driven by the validator; the query phase runs on demand and is driven by the user. They share only the database.

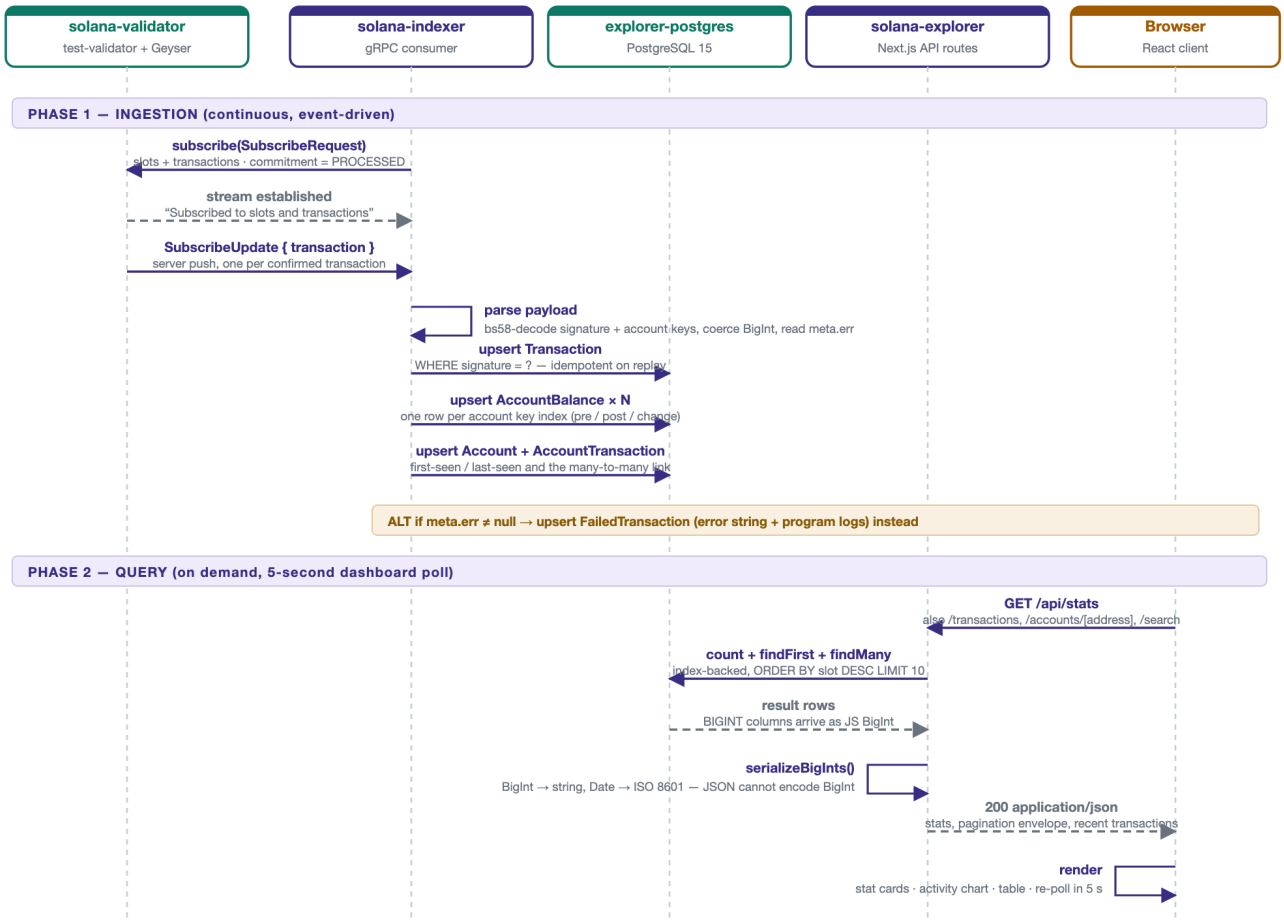


Figure 4.3 — End-to-end sequence. Phase 1 is push-driven ingestion; phase 2 is the on-demand query path taken by every page and API call.

4.4.1 Per-message processing

Each `SubscribeUpdate` carrying a transaction is processed as follows:

1. **Signature extraction.** The 64-byte signature buffer is base-58 encoded. Values that are already strings pass through; anything else degrades to `"unknown"` rather than throwing, so one malformed message cannot kill the stream.
2. **Account key extraction.** Each 32-byte public key is base-58 encoded, with array order preserved because it is semantically load-bearing (D-6).
3. **Instruction extraction.** For each instruction, the program index, the account index list and the base-58 encoded data are captured together with the ordinal position.
4. **Metadata extraction.** Fee and compute units are coerced to `BigInt`; success is determined by whether the metadata error is null; the pre- and post-balance arrays and the program logs are read.
5. **Branch.** Successful transactions are upserted into `Transaction`, followed by their balance rows and their account links. Failed transactions are upserted into `FailedTransaction` with the serialised error and logs.
6. **Error containment.** Unique-constraint violations (`P2002`) are swallowed as benign; any other error is logged with the signature and the loop continues to the next message.

4.5 API Design

Five read-only endpoints cover every screen and every documented programmatic use. They share three conventions: 64-bit values are strings, list responses carry a uniform `pagination` envelope, and every route is marked `force-dynamic` so that a build-time render never freezes live data.

METHOD	ROUTE	RETURNS
GET	`/api/stats`	Total transactions, total failures, success rate to one decimal, latest slot, and the ten most recent transactions.
GET	`/api/transactions?page&limit`	Transactions ordered by descending slot, plus `{page, limit, total, totalPages}`. Page defaults to 1, limit to 20, clamped to 100.
GET	`/api/transactions/{signature}`	A single transaction with its memos; on miss, the matching failed transaction with `success: false`; otherwise 404.
GET	`/api/accounts/{address}`	Account record, latest balance, aggregate net change, and a paginated balance history with the linked transaction. Accepts `field`, `unit`, `amount`, `minAmount` and `maxAmount`. 404 if the address is unknown.
GET	`/api/search?q`	Up to three matching signatures and three matching addresses by prefix; an empty result set below eight characters.

Table 4.3 — REST endpoint summary. Full request and response shapes are in Appendix A.

4.5.1 Input handling contract

All query parameters are treated as untrusted. Page and limit are parsed, floored at one and clamped at their maxima. The balance filter's `field` parameter is validated against an allow-list of three column names and silently falls back to `postBalance` if unrecognised — the parameter never reaches the query builder as free text. Amounts are parsed with an explicit finiteness check, so an empty or non-numeric value is dropped rather than coerced to zero, and SOL values are converted to lamport `BigInt`s before comparison so that filtering happens in the database's own integer domain.

4.6 User Interface Design

The interface is organised around a single navigational spine: everything is reachable from the dashboard in at most two clicks, and every identifier rendered anywhere is a link to its own detail page.

ROUTE	PURPOSE AND PRINCIPAL COMPONENTS
`/`	**Dashboard.** Four stat cards (total, failed, success rate, latest slot), a stacked success/failure activity chart with 5 m / 15 m / 1 h / all ranges, and a recent-transactions table. Polls `/api/stats` every five seconds and updates in place.
`/transactions`	**Transaction list.** Paginated table with signature, slot, status, fee in SOL and relative time; page state lives in the URL so a view can be shared.
`/transactions/[signature]`	**Transaction detail.** Status, slot, fee, compute units; the ordered account list with each address linked to its account page; the instruction list; memos; and, for failures, the error and the scrollable program log.
`/accounts/[address]`	**Account detail.** Address with copy control, first/last seen, current balance, and the filterable balance-history table with signed, colour-coded net change per row.
Global (navbar)	**⌘K search dialog.** Debounced prefix search across signatures and addresses; selection routes to the corresponding detail page. Also hosts the light/dark/system theme control.

Three interaction principles were applied consistently. *Loading is never a blank screen*: every asynchronous region has a skeleton matching its final layout. *Truncation never hides data*: shortened signatures carry a tooltip with the full value and a copy button. *State is addressable*: pagination and filters are query parameters, so any view can be bookmarked, shared or reloaded without loss.

4.7 Deployment Design

Deployment is a single Compose file describing four services, two named volumes and an explicit readiness-gated start-up order. Figure 4.4 shows the resulting dependency graph.

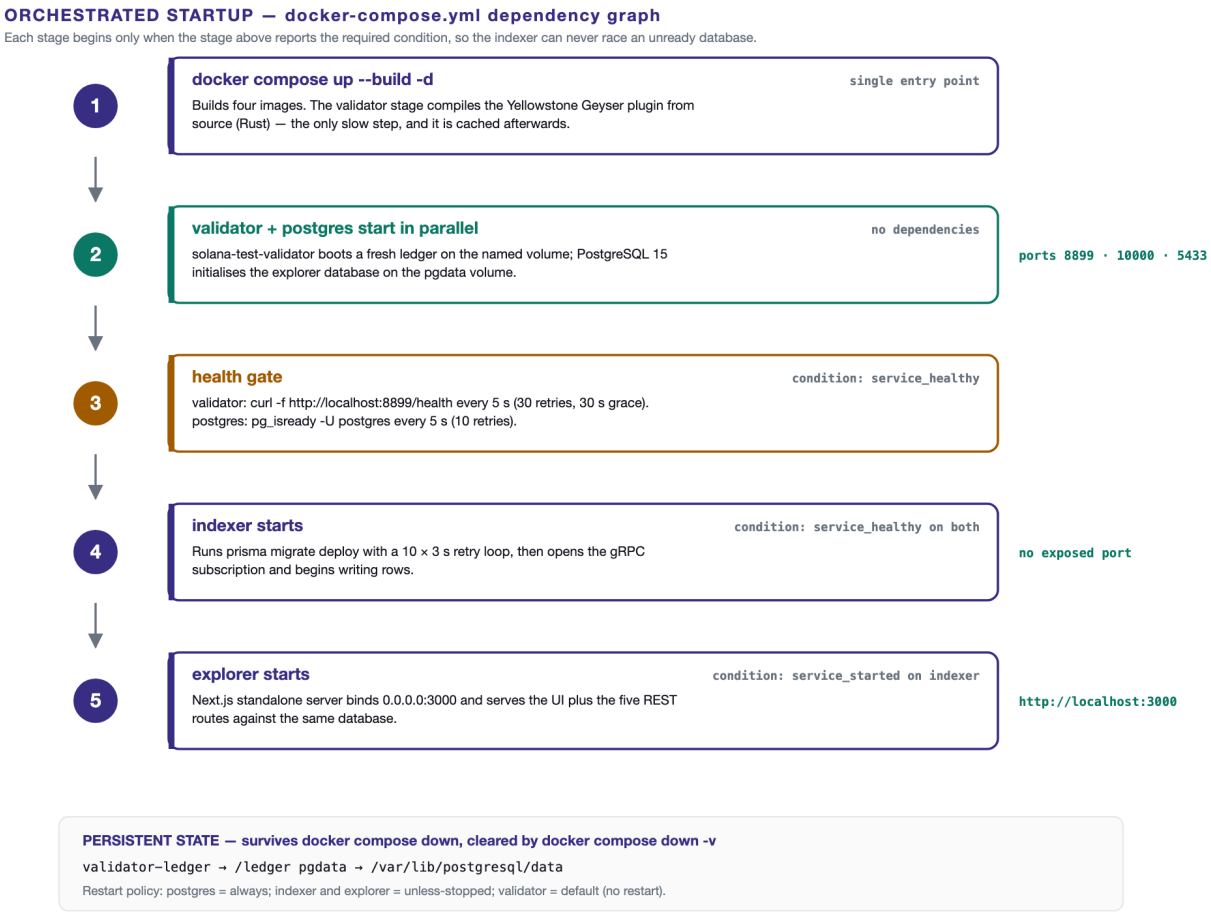


Figure 4.4 — Orchestrated start-up. Each stage is gated on the previous stage reporting the stated condition.

The important property is that the ordering is expressed in terms of *readiness*, not merely process start. The validator is healthy only once its RPC endpoint answers `/health`; PostgreSQL is healthy only once `pg_isready` succeeds. The indexer waits on both, and only then runs migrations and subscribes. This is what makes the one-command promise of FR-22 hold on a cold machine, where the validator's first boot takes appreciably longer than the database's.

CHAPTER 5

Implementation

5.1 Technology Stack

LAYER	TECHNOLOGY	VERSION	WHY THIS CHOICE
Blockchain	solana-test-validator	`1.18.26`	Full single-node cluster with instant finality and unlimited airdrops; the standard local development target.
Streaming	Yellowstone gRPC Geyser plugin	`1.15.3`	Production-grade Geyser implementation with a stable Protobuf contract; the version tag is pinned to match the validator ABI.
Indexer runtime	Node.js + TypeScript	`22 / 5.9`	Shares a language and a generated Prisma client with the explorer; static types catch wire-shape mistakes at compile time.
gRPC client	@triton-one/yellowstone-grpc	`4.0.2`	Official typed client for the plugin; removes hand-rolled Protobuf loading from the hot path.
Encoding	bs58	`5.0`	Base-58 encoding for signatures, public keys and instruction data.
ORM	Prisma	`5.22`	Declarative schema, generated type-safe client, first-class `BigInt` mapping, and a migration system checked into the repository.
Database	PostgreSQL	`15`	64-bit integers, native arrays and JSONB, real secondary indexes, server-side aggregation.
Web framework	Next.js (App Router)	`16.1.6`	Single deployable serving both the interface and the REST routes; standalone output keeps the runtime image small.
UI runtime	React	`19.2`	Component model with concurrent rendering for the polled dashboard.
Styling	Tailwind CSS + shadcn/ui	`4 / —`	Utility-first styling with an accessible, unstyled-primitive component layer (Radix) rather than an opinionated component library.
Charts	Recharts	`3.8`	Declarative composable charts; used for the stacked success/failure activity area chart.
Motion	Framer Motion	`12.40`	Staggered table and card entrances, with `useReducedMotion` respected throughout.
Testing	Vitest	`4.1.11`	Fast, zero-config TypeScript test runner used identically in both packages.
Orchestration	Docker Compose	`v2`	Declarative multi-container topology with health checks and named volumes; the mechanism behind the one-command promise.
Package manager	pnpm	`10.27`	Deterministic installs from a committed lockfile; content-addressed store makes Docker layer caching effective.

Table 5.1 — Technology stack with versions and justification.

5.2 Repository Layout

```
Solana-Local-Indexer-Explorer/
├── docker-compose.yml          orchestration: 4 services, 2 volumes, health gates
├── README.md                   user manual / quick start
├── INSTALLATION_GUIDE.md       first-time setup walkthrough
├── TEST_CASES_AND_VALIDATION.md  full test catalogue and results
│
├── validator/
│   ├── Dockerfile              2-stage: build Geyser plugin (Rust) → runtime with Solana CLI
│   └── plugin-config.json      Geyser plugin config: libpath, gRPC bind address, capacities
│
├── indexer/
│   ├── Dockerfile
│   ├── prisma/
│   │   ├── schema.prisma       6 models, indexes, unique constraints
│   │   └── migrations/         0_init · add_accounts · add_account_balances
│   └── src/
│       ├── main.ts             entrypoint: migrate with retry, then start the indexer
│       ├── indexer.ts          gRPC subscription and the per-message handler
│       ├── db/client.ts        Prisma singleton
│       ├── db/query.ts         upsertTransaction · upsertFailedTransaction · upsertAccounts…
│       └── grpc/
│           ├── parse-transaction.test.ts         29 unit tests
│           └── parse-transaction.test-helper.ts  parsing logic under test
│
└── explorer/
    ├── Dockerfile              deps → build → standalone runner
    ├── prisma/schema.prisma    same schema, read-only client
    └── src/
        ├── app/
        │   ├── page.tsx                      dashboard
        │   ├── transactions/page.tsx         list
        │   ├── transactions/[signature]/     detail
        │   ├── accounts/[address]/           account detail + balance filter
        │   └── api/                          stats · transactions · accounts · search
        ├── components/                       stat-card · data-table · transaction-chart · navbar
…
        └── lib/
            ├── serialization.ts              serializeBigInts()
            ├── serialization.test.ts         9 unit tests
            ├── prisma.ts                     Prisma singleton
            └── types.ts                      shared response types
```

5.3 The Validator Image

The validator image is a two-stage build. The first stage starts from `rust:1.76`, installs the native toolchain the plugin needs (`clang`, `cmake`, `protobuf-compiler`, OpenSSL headers), clones the Yellowstone repository, checks out a tag that matches the validator's ABI, and compiles only the Geyser package:

```
# Pin the plugin to a tag built against Solana 1.18.26, with a fallback
RUN git clone https://github.com/rpcpool/yellowstone-grpc.git && \
    cd yellowstone-grpc && \
    git checkout v1.15.3+solana.1.18.26 || git checkout v1.15.3+solana.1.18.25

WORKDIR /build/yellowstone-grpc
RUN cargo build --release --package yellowstone-grpc-geyser
```

The second stage is a slim Debian runtime that installs the Solana CLI release, copies in the single compiled artefact `libyellowstone_grpc_geyser.so` and the plugin configuration, and starts the validator with the plugin attached:

```
COPY --from=builder /build/yellowstone-grpc/target/release/libyellowstone_grpc_geyser.so \
     /plugins/libyellowstone_grpc_geyser.so
COPY plugin-config.json /config/plugin-config.json
EXPOSE 8899 10000
CMD ["solana-test-validator", "--ledger", "/ledger", \
     "--geyser-plugin-config", "/config/plugin-config.json"]
```

The plugin configuration binds gRPC to `0.0.0.0:10000` so the indexer container can reach it, sets channel capacities appropriate for a local validator, and sets `block_fail_action` to `log` so that a plugin problem is reported rather than halting block production. The two-stage split matters practically: the Rust toolchain and the ~1 GB build tree never enter the final image, and the expensive compile is cached after the first build.

5.4 The Indexer Service

5.4.1 Start-up and migration

`main.ts` establishes the schema before any data can arrive. Compose health checks confirm that PostgreSQL is accepting connections, but not that it has finished first-time initialisation, so migration is wrapped in a bounded retry loop (decision D-10):

```
async function runMigrations(retries = 10, delayMs = 3000) {
  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      execSync("pnpm prisma migrate deploy", { stdio: "inherit" });
      return;
    } catch (error) {
      if (attempt === retries) process.exit(1);
      await sleep(delayMs);
    }
  }
}
```

Exiting non-zero after ten attempts is deliberate: combined with `restart: unless-stopped`, a genuinely unreachable database produces a visible restart loop and a clear log rather than a silently idle container.

5.4.2 Subscription

The endpoint is normalised before use, because the client requires a full URL while operators naturally write `host:port` (FR-2):

```
let endpoint = YELLOWSTONE_ADDR;
if (!endpoint.startsWith("http://") && !endpoint.startsWith("https://")) {
  endpoint = `http://${endpoint}`;
}
const client = new Client(endpoint, YELLOWSTONE_XTOKEN || undefined, {
  "grpc.max_receive_message_length": 64 * 1024 * 1024,   // 64 MB
});
```

The 64 MB receive limit replaces the client default, which is too small for busy slots and otherwise causes intermittent stream termination under load. The subscription itself requests slots and all transactions, with empty include/exclude/require lists meaning “no filter”:

```
const request: SubscribeRequest = {
  accounts: {}, slots: { client: { filterByCommitment: false } },
  transactions: { client: { accountInclude: [], accountExclude: [], accountRequired: [] } },
  transactionsStatus: {}, entry: {}, blocks: {}, blocksMeta: {},
  commitment: CommitmentLevel.PROCESSED,
  accountsDataSlice: [], ping: undefined,
};
```

5.4.3 Payload parsing

Parsing is the layer where the wire format meets the domain model, and it is written defensively because a single malformed message must not terminate a long-lived stream. Each extractor accepts a buffer, a typed array or an already-decoded string, and degrades to a safe default rather than throwing:

```
if (transaction.signature instanceof Uint8Array ||
    Buffer.isBuffer(transaction.signature)) {
  signature = bs58.encode(transaction.signature);
} else if (typeof transaction.signature === "string") {
  signature = transaction.signature;
} else {
  signature = "unknown";
}
```

Account keys are decoded in order; instructions are captured with their program index, account indices, base-58 data and ordinal position; fee and compute units are coerced to `BigInt`; and success is decided by `meta.err === null || meta.err === undefined`. Because this logic is pure — buffers in, plain values out — it is exercised by 29 unit tests without a validator or a database (NFR-10); the suite is described in Section 6.3.

5.4.4 Persistence

Writes go through `db/query.ts`. The transaction upsert is keyed on the unique signature, and the balance rows are written in the same call, one per account key index, each upserted on the composite key so that re-processing overwrites in place:

```
const balancePromises = data.accounts.map(async (address, index) => {
  if (index < data.preBalances.length && index < data.postBalances.length) {
    const preBalance  = BigInt(data.preBalances[index]);
    const postBalance = BigInt(data.postBalances[index]);
    const balanceChange = postBalance - preBalance;

    return prisma.accountBalance.upsert({
      where:  { transactionId_accountIndex: { transactionId: tx.id, accountIndex: index } },
      update: { preBalance, postBalance, balanceChange },
      create: { accountAddress: address, transactionId: tx.id, accountIndex: index,
                preBalance, postBalance, balanceChange,
                slot: data.slot, blockTime: data.blockTime },
    });
  }
});
await Promise.all(balancePromises);
```

Two details are worth noting. The index bounds check guards against the balance arrays being shorter than the account key array, which the wire format permits. And `balanceChange` is computed once at write time in `BigInt` arithmetic and stored, so the amount filter of FR-17 becomes a plain indexed comparison rather than a computed expression (decision D-6).

Account links are then written by `upsertAccountsForTransaction`, which upserts the `Account` row — an empty `update` is intentional, because `@updatedAt` refreshes `lastSeen` regardless — and then upserts the join row on its composite key. Finally, the handler swallows Prisma's `P2002` unique-violation code as benign and logs anything else, so that a duplicate delivery is a no-op and a real fault is visible in `docker compose logs indexer` (NFR-9).

5.5 Database and Migrations

The schema lives in `indexer/prisma/schema.prisma` and is applied through three checked-in migrations, which together form an auditable history of how the data model evolved:

MIGRATION	CONTENTS
`0_init`	`Transaction`, `Memo` and `FailedTransaction` with their unique constraints and secondary indexes.
`20260320121218_add_accounts`	`Account` and `AccountTransaction`, the composite unique link constraint, and both foreign keys.
`20260421205823_add_account_balances`	`AccountBalance`, its three indexes, the `(transactionId, accountIndex)` unique constraint and the cascading foreign key.

Because `prisma migrate deploy` applies only unapplied migrations, an existing installation upgrades in place: a user who indexed data before balance tracking existed keeps their transactions and starts accumulating balance rows from the next transaction onward. The explorer package carries a copy of the same schema solely to generate its read-only client; the indexer remains the single owner of schema evolution (D-10).

5.6 The Explorer API Layer

Each route is a small App Router handler. The pattern is consistent: parse and clamp input, issue the queries concurrently, then serialise. The transaction list is representative:

```
const page  = Math.max(1, parseInt(searchParams.get("page")  || "1", 10));
const limit = Math.min(100, Math.max(1, parseInt(searchParams.get("limit") || "20", 10)));
const skip  = (page - 1) * limit;

const [transactions, total] = await Promise.all([
  prisma.transaction.findMany({
    orderBy: { slot: "desc" }, skip, take: limit,
    select: { id: true, signature: true, slot: true,
              blockTime: true, success: true, fee: true },
  }),
  prisma.transaction.count(),
]);

return NextResponse.json(serializeBigInts({
  transactions,
  pagination: { page, limit, total, totalPages: Math.ceil(total / limit) },
}));
```

The explicit `select` keeps the large `accounts` and `instructions` columns out of list responses; the count runs concurrently with the page query rather than after it; and the clamp on `limit` enforces NFR-5 at the boundary.

5.6.1 BigInt serialisation

Every response passes through one recursive helper. It is fifteen lines long and is the single point at which the pipeline's 64-bit values become JSON-safe (D-8, FR-19):

```
export function serializeBigInts<T>(obj: T): T {
  if (obj === null || obj === undefined) return obj;
  if (typeof obj === "bigint") return obj.toString() as unknown as T;
  if (obj instanceof Date)     return obj.toISOString() as unknown as T;
  if (Array.isArray(obj))      return obj.map(serializeBigInts) as unknown as T;
  if (typeof obj === "object") {
    const result: Record<string, unknown> = {};
    for (const [key, value] of Object.entries(obj)) result[key] = serializeBigInts(value);
    return result as T;
  }
  return obj;
}
```

Two behaviours are load-bearing and are pinned by tests: ordinary numbers must pass through *unchanged*, so that `pagination.page` stays a number while `slot` becomes a string; and negative values must survive, because `balanceChange` is negative for every account that pays a fee.

5.6.2 The account endpoint and its filter

The account route is the most involved, because it serves a filtered, paginated, aggregated view. The filter is built from validated inputs and then reused across all four queries so that the table, the total, the aggregate and the pagination always agree:

```
const AMOUNT_FIELDS = new Set(["postBalance", "preBalance", "balanceChange"]);
const field = AMOUNT_FIELDS.has(requestedField) ? requestedField : "postBalance";
const unit  = searchParams.get("unit") === "lamports" ? "lamports" : "sol";

const amountFilter: Record<string, bigint> = {};
if (exact !== null) amountFilter.equals = exact;
else {
  if (min !== null) amountFilter.gte = min;
  if (max !== null) amountFilter.lte = max;
}
const where: Record<string, unknown> = { accountAddress: address };
if (Object.keys(amountFilter).length > 0) where[field] = amountFilter;
```

The `latestBalance` query deliberately ignores the filter and always reads the newest row for the address, because a user filtering the history still expects the headline balance to reflect reality. Unit conversion happens once, on the way in:

```
function parseAmount(value: string | null, unit: string): bigint | null {
  if (value === null || value.trim() === "") return null;
  const num = Number(value);
  if (!Number.isFinite(num)) return null;
  if (unit === "lamports") return BigInt(Math.round(num));
  return BigInt(Math.round(num * LAMPORTS_PER_SOL));
}
```

5.7 The Explorer User Interface

The dashboard is a client component that polls on an interval and cleans up after itself, with the loading flag cleared only once so that subsequent refreshes update in place rather than flashing a skeleton:

```
useEffect(() => {
  fetchStats();
  const interval = setInterval(fetchStats, 5000);
  return () => clearInterval(interval);
}, []);
```

The activity chart takes the hundred most recent transactions and buckets them client-side into twenty-four intervals sized to the selected range, counting successes and failures separately so that Recharts can stack them. Bucketing on the client rather than in SQL keeps range switching instantaneous and avoids adding a group-by endpoint that only one component would use.

Global search is a `cmdk` dialog bound to ⌘K / Ctrl-K. Input is debounced by 300 ms before the request is issued, and the effect returns a cleanup that clears the pending timer, so a fast typist produces one request rather than one per keystroke.

Presentational conventions are shared across pages through small components: `StatCard`, `StatusBadge`, `CopyButton`, `DataTableShell`, `PaginationControls` and a set of skeletons. Lamport values are rendered as SOL to nine decimal places with tabular figures, so that columns of balances align on the decimal point.

5.8 Containerisation and Orchestration

The explorer image is a three-stage build — dependencies, build, runtime — that ends with Next.js standalone output, so the runtime image carries a server bundle and static assets rather than a full `node_modules` tree. Both Node images enable `corepack` and install with `--frozen-lockfile` against a cache-mounted pnpm store, so repeat builds reuse the dependency layer.

The Compose file expresses the readiness gates described in Section 4.7 directly:

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8899/health"]
  interval: 5s
  retries: 30
  start_period: 30s
...
indexer:
  depends_on:
    validator: { condition: service_healthy }
    postgres:  { condition: service_healthy }
  environment:
    DATABASE_URL: postgresql://postgres:password@postgres:5432/explorer
    YELLOWSTONE_ADDR: validator:10000
  restart: unless-stopped
```

Because `YELLOWSTONE_ADDR` is an ordinary environment variable, UC-9 requires no file edit at all:

```
YELLOWSTONE_ADDR=your-grpc-host:10000 YELLOWSTONE_XTOKEN=your-token \
  docker compose up -d
```

CHAPTER 6

Testing and Validation

6.1 Test Strategy

Testing was organised around a simple principle: **automate what can be verified without infrastructure, and script precisely what cannot**. The two layers where a silent bug would corrupt data — gRPC payload parsing and 64-bit serialisation — are pure functions, so they are covered by an automated suite that runs in under a second with no validator and no database. Everything above them depends on a live four-container stack, so those cases are written as reproducible procedures with explicit commands, preconditions and expected results.

> **HOW TO READ THE RESULT COLUMNS**
>
> Cases marked AUTOMATED were executed by the authors with `pnpm test` on **28 August 2026**; the verbatim console output is reproduced in Appendix D and can be regenerated by anyone in seconds. Cases marked MANUAL are scripted procedures exercised against the running stack during development; they are documented here in full so that an examiner can reproduce each one independently rather than take the result on trust. Only the automated results constitute machine-checkable evidence.

LEVEL	METHOD	TOOLING	CASES	WHAT IT PROTECTS
Unit — parsing	AUTOMATED	`Vitest`	29	Correct decoding of every field of the gRPC transaction payload, including malformed and missing input.
Unit — serialisation	AUTOMATED	`Vitest`	9	Lossless conversion of `BigInt` and `Date` to JSON, with ordinary numbers untouched.
Integration	MANUAL	`docker · psql`	12	Container health, start-up ordering, migration, stream subscription, end-to-end persistence.
API	MANUAL	`curl · jq`	18	Endpoint contracts, pagination, filtering, search, error codes and JSON encoding.
Interface	MANUAL	`browser`	16	Navigation, live refresh, detail rendering, filter behaviour, empty and error states.
Non-functional	MANUAL	`mixed`	8	Latency, restart behaviour, idempotence, portability, reset and configurability.
Total			**92**	**38 automated · 54 documented manual procedures**

Table 6.1 — Test levels, tooling and coverage.

6.2 Test Environment

HOST	Apple Silicon macOS workstation; validator running under `linux/amd64` emulation
CONTAINER RUNTIME	Docker Desktop 24.x with Compose V2
AUTOMATED RUNNER	Vitest 4.1.11 on Node.js 22, pnpm 10.27
SERVICES UNDER TEST	`solana-validator` · `explorer-postgres` · `solana-indexer` · `solana-explorer`
DATABASE ACCESS	`postgresql://postgres:password@localhost:5433/explorer`
EXPLORER UNDER TEST	`http://localhost:3000`

TEST DATA	Validator vote transactions produced continuously by the test validator, plus deliberate transfers created with the Solana CLI inside the validator container
DATE OF AUTOMATED RUN	28 August 2026

6.3 Unit Test Results

38	38	0	2	<1 s
AUTOMATED CASES	PASSED	FAILED	TEST FILES	SUITE RUNTIME

6.3.1 Explorer — `serializeBigInts()`

Nine cases cover the function's entire behavioural surface: pass-through, conversion, recursion and the two edge values (negative and zero) that a naive truthiness check would mishandle.

ID	CASE	EXPECTED RESULT	STATUS
UT-SER-01	Returns null / undefined unchanged	Null and undefined pass through without conversion or error	PASS
UT-SER-02	Converts a top-level bigint to a string	`123n → "123"`	PASS
UT-SER-03	Converts a Date to an ISO string	`"2026-01-01T00:00:00.000Z"`	PASS
UT-SER-04	Leaves primitives unchanged	Number, string and boolean are returned identically and keep their types	PASS
UT-SER-05	Recursively converts bigints in a nested object	Every bigint at any depth becomes a string; other keys untouched	PASS
UT-SER-06	Recursively converts bigints inside an array	Each element is processed independently and order is preserved	PASS
UT-SER-07	Realistic API payload (transaction list + pagination)	`slot` and `fee` become strings, `blockTime` becomes ISO, and `pagination.page` remains a number	PASS
UT-SER-08	Negative bigint (balanceChange can be negative)	`-10000n → "-10000"`	PASS
UT-SER-09	Zero as a bigint	`0n → "0"`, not dropped as falsy	PASS

Table 6.2 — Explorer serialisation unit tests (`explorer/src/lib/serialization.test.ts`).

6.3.2 Indexer — gRPC payload parsing

Twenty-nine cases cover each extractor with a valid buffer, a typed array, an already-decoded string, a missing field and — where relevant — a value of an unexpected type, plus one end-to-end case over a complete realistic payload.

ID	UNIT	CASE	EXPECTED RESULT	STATUS
UT-PRS-01	`parseSignature`	Buffer signature	Base-58 encoding of the 64-byte buffer	PASS
UT-PRS-02	`parseSignature`	Uint8Array signature	Identical result to the Buffer case	PASS
UT-PRS-03	`parseSignature`	Already-decoded string	Returned unchanged, not re-encoded	PASS
UT-PRS-04	`parseSignature`	Missing (null / undefined)	`"unknown"`, no exception	PASS

ID	UNIT	CASE	EXPECTED RESULT	STATUS
UT-PRS-05	parseSignature	Unexpected type (number)	"unknown", no exception	PASS
UT-PRS-06	parseAccounts	Buffer account keys	Array of base-58 strings	PASS
UT-PRS-07	parseAccounts	String account keys	Returned unchanged	PASS
UT-PRS-08	parseAccounts	Missing accountKeys	Empty array, no exception	PASS
UT-PRS-09	parseAccounts	Order preservation	Input order preserved — the index is the balance-row key	PASS
UT-PRS-10	parseInstructions	Full instruction	Program index, account list, base-58 data and ordinal all captured	PASS
UT-PRS-11	parseInstructions	Missing accounts field	Defaults to an empty array	PASS
UT-PRS-12	parseInstructions	Instruction with no data	data is null rather than undefined or ""	PASS
UT-PRS-13	parseInstructions	Multiple instructions	Ordinals are 0, 1, 2 matching array position	PASS
UT-PRS-14	parseInstructions	Missing instructions	Empty array, no exception	PASS
UT-PRS-15	parseComputeUnitsUsed	Value present	2100 → 2100n	PASS
UT-PRS-16	parseComputeUnitsUsed	Value missing	undefined	PASS
UT-PRS-17	parseComputeUnitsUsed	Value is 0 (falsy but valid)	undefined — documents defect D-06 (Section 6.8) rather than asserting desired behaviour	PASS
UT-PRS-18	parseFee	Fee present	5000 → 5000n	PASS
UT-PRS-19	parseFee	Fee missing	undefined	PASS
UT-PRS-20	parseSuccess	meta.err === null	true	PASS
UT-PRS-21	parseSuccess	meta.err absent	true — absence is not failure	PASS
UT-PRS-22	parseSuccess	Populated error object	false	PASS
UT-PRS-23	parseError	No error	null	PASS
UT-PRS-24	parseError	Error object present	Serialised with JSON.stringify for storage as text	PASS
UT-PRS-25	parseBalances	Both arrays present	Pre- and post-balance arrays extracted intact	PASS
UT-PRS-26	parseBalances	Arrays missing	Two empty arrays, so the balance loop is skipped safely	PASS
UT-PRS-27	parseLogs	Log messages present	Array returned intact and in order	PASS
UT-PRS-28	parseLogs	Log messages missing	Empty array	PASS

ID	UNIT	CASE	EXPECTED RESULT	STATUS
UT-PRS-29	end-to-end	Complete realistic payload	All ten fields parse consistently from one vote-transaction-shaped message	PASS

Table 6.3 — Indexer parsing unit tests (`indexer/src/grpc/parse-transaction.test.ts`).

> **A NOTE ON THE TEST HELPER**
>
> The parsing logic lives inline in the `stream.on("data", …)` handler in `indexer.ts`. To make it testable without restructuring the production hot path mid-project, the suite exercises a documented mirror of that logic in `parse-transaction.test-helper.ts`, which carries a header warning that it must be updated in step with the handler. This is an acknowledged trade-off: it buys coverage now at the cost of a manual synchronisation obligation. Extracting the handler into exported functions and pointing the tests at the real implementation is listed as future work in Section 9.2.

6.4 Integration Test Cases

ID	OBJECTIVE	PROCEDURE	EXPECTED RESULT
IT-01	Clean build from source	`docker compose up --build -d` on a machine with no cached layers	All four images build; the validator stage compiles the Geyser plugin without error; no manual intervention required.
IT-02	All services reach running state	`docker compose ps`	Four rows `Up`; `solana-validator` and `explorer-postgres` additionally show `(healthy)`.
IT-03	Validator RPC responds	`curl -X POST localhost:8899 -d '{"jsonrpc":"2.0","id":1,"method":"getSlot"}'`	HTTP 200 with a JSON body containing a numeric `result` that increases on repeat calls.
IT-04	Health gate is honoured	Inspect start-up order in `docker compose logs`	The indexer's first log line appears only after both health checks pass; no connection-refused errors precede it.
IT-05	Migrations apply on first start	`docker compose logs indexer | head`	"Running database migrations…" followed by "Migrations completed"; all three migrations applied.
IT-06	Migration retry under a slow database	Start the stack on a cold machine where PostgreSQL initialises slowly	The indexer logs a retry and succeeds on a later attempt rather than exiting; the container does not enter a crash loop.

ID	OBJECTIVE	PROCEDURE	EXPECTED RESULT
IT-07	gRPC subscription established	`docker compose logs indexer`	"Connecting to Yellowstone gRPC: http://validator:10000" followed by "Subscribed to slots and transactions".
IT-08	Slot updates received	Observe indexer logs for 30 seconds	Continuous `Slot <n> status: <s>` lines with a monotonically increasing slot number.
IT-09	End-to-end transaction persistence	Airdrop and transfer inside the validator container, then query the database	A row with the new signature appears in `Transaction`; the indexer logs `Indexed transaction <prefix>…`.
IT-10	Balance rows written per account index	`SELECT * FROM "AccountBalance" WHERE "transactionId" = …`	One row per account key, with `accountIndex` matching wire order and `balanceChange = postBalance − preBalance`.
IT-11	Account registry and links populated	Query `Account` and `AccountTransaction`	Every address in the transaction has an `Account` row and exactly one join row per transaction.
IT-12	Failed transaction captured separately	Submit a transaction that fails (for example, transfer more than the balance)	A row appears in `FailedTransaction` with the serialised error and the program logs; `Transaction` is unaffected; the dashboard's failed count increments.

Table 6.4 — Integration test cases.

6.5 API Test Cases

ID	REQUEST	EXPECTED RESPONSE
API-01	`GET /api/stats`	200; body contains `totalTransactions`, `totalFailed`, `successRate`, `latestSlot` and a `recentTransactions` array of at most 10.
API-02	`GET /api/stats`	`successRate` equals `total ÷ (total + failed) × 100` rounded to one decimal; equals 0 when both counts are 0 (no division by zero).
API-03	`GET /api/stats`	`latestSlot` is a decimal string, not a number, and equals the maximum indexed slot.
API-04	`GET /api/transactions`	200; 20 items by default; `pagination` reports `page 1`, `limit 20` and a correct `totalPages`.
API-05	`GET /api/transactions?limit=5&page=2`	Exactly 5 items, disjoint from page 1, ordered by descending slot.
API-06	`GET /api/transactions?limit=5000`	Limit is clamped to 100; the response never exceeds 100 items.

ID	REQUEST	EXPECTED RESPONSE
API-07	`GET /api/transactions?page=0&limit=0`	Page floors to 1 and limit floors to 1; no error and no empty page.
API-08	`GET /api/transactions?page=abc`	Unparseable values fall back to defaults rather than producing `NaN` in the query.
API-09	`GET /api/transactions/{valid}`	200; full record including `accounts`, `instructions` and `memos`; `slot` and `fee` are strings.
API-10	`GET /api/transactions/{failed}`	200; the failed record is returned with `success: false`, the error string and the log array.
API-11	`GET /api/transactions/{unknown}`	404 with `{"error":"Transaction not found"}`.
API-12	`GET /api/accounts/{known}`	200; account record, `latestBalance`, `totalBalanceChange`, a balance array and a pagination envelope.
API-13	`GET /api/accounts/{unknown}`	404 with `{"error":"Account not found"}`.
API-14	`…?field=postBalance&amount=1&unit=sol`	Only rows whose post-balance is exactly 1 000 000 000 lamports; `filter` echoes the applied criteria; count and pagination reflect the filtered set.
API-15	`…?minAmount=0.1&maxAmount=10`	Inclusive range applied to the selected field; an exact `amount`, if also supplied, takes precedence over the range.
API-16	`…?field=DROP TABLE`	Unrecognised field silently falls back to `postBalance`; the value never reaches the query builder.
API-17	`GET /api/search?q=abcdefgh`	200; up to three signature matches and three address matches by prefix, each tagged with its type.
API-18	`GET /api/search?q=abc`	200 with an empty `results` array — the eight-character minimum is enforced server-side, not only in the UI.

Table 6.5 — API test cases.

6.6 User Interface Test Cases

ID	SCENARIO	EXPECTED BEHAVIOUR
UI-01	Dashboard first paint	Skeletons matching the final layout are shown, then replaced by the four stat cards, the chart and the recent-transactions table.
UI-02	Dashboard live refresh	Values update roughly every five seconds with no full-page reload and no skeleton flash on subsequent refreshes.
UI-03	Backend unavailable	The dashboard renders an explicit “Failed to load stats data” panel rather than an empty page or a stack trace.
UI-04	Activity chart ranges	Switching 5 m / 15 m / 1 h / All re-buckets client-side and immediately; successes and failures are stacked and separately coloured.
UI-05	Chart with no data	An explanatory empty state is shown instead of an axis with no series.
UI-06	Transaction list pagination	Next / previous update both the table and the URL; a reload restores the same page.
UI-07	Signature truncation	Truncated signatures reveal the full value on hover and can be copied with the inline control.
UI-08	Navigate to transaction detail	Selecting a signature routes to `/transactions/{sig}` and renders status, slot, fee, compute units, accounts and instructions.

ID	SCENARIO	EXPECTED BEHAVIOUR
UI-09	Failed transaction detail	The error is displayed prominently and the program logs are rendered in a scrollable monospace panel.
UI-10	Unknown signature	A “Transaction not found” state with a route back to the transaction list; no unhandled error.
UI-11	Account link from a transaction	Each account in the account list links to `/accounts/{address}` and navigates correctly.
UI-12	Account detail rendering	Address, first seen, last seen, current balance in SOL to nine decimals, and the paginated balance history are all shown.
UI-13	Balance change formatting	Positive changes are prefixed with + and coloured as gains; negative changes are coloured as losses; zero is neutral.
UI-14	Balance filter round trip	Applying a filter updates the URL; reloading preserves it; “Clear” returns to the unfiltered view.
UI-15	⌘K search	The dialog opens on ⌘K / Ctrl-K, debounces input, lists matches grouped by type, and routes to the chosen record.
UI-16	Theme switching	Light, dark and system themes apply immediately across every page and persist across reloads.

Table 6.6 — User interface test cases.

6.7 Non-Functional Test Cases

ID	ATTRIBUTE	PROCEDURE	ACCEPTANCE CRITERION
NF-01	Usability (NFR-1)	A user with only Docker follows the README Quick Start	A working explorer is reached with one command and no undocumented prerequisite.
NF-02	Latency (NFR-2)	Send a transfer and watch the dashboard	The transaction appears within one five-second refresh cycle.
NF-03	Reliability (NFR-3)	`docker restart solana-indexer`	The container restarts, re-runs migrations as a no-op, re-subscribes and resumes indexing without operator action.
NF-04	Idempotence (NFR-4)	Restart the indexer and let the stream redeliver	No duplicate signatures: `SELECT signature, COUNT(*) … HAVING COUNT(*) > 1` returns no rows.
NF-05	Precision (NFR-4)	Compare a large lamport balance in the API response with the database value	Exact match; the value is a string in JSON and no digits are lost.
NF-06	Portability (NFR-6)	Build and run on Apple Silicon under `linux/amd64` emulation	The stack builds and runs unmodified; only build time is affected.
NF-07	Reset (FR-22)	`docker compose down -v` then `up -d`	Both named volumes are removed; the next start yields a fresh ledger and an empty, freshly migrated database.
NF-08	Configurability (NFR-8)	Start with `YELLOWSTONE_ADDR` pointing at an external endpoint	The indexer connects to the external endpoint — confirmed in the log line — and indexes that chain instead, with no code change.

Table 6.7 — Non-functional test cases.

6.8 Defects Found and Resolved

The following defects were found during development and testing. Eight were fixed; two are documented open items with known workarounds, recorded here rather than omitted.

ID	DEFECT	SEVERITY	ROOT CAUSE AND RESOLUTION	STATE
D-01	`JSON.stringify` throws on Prisma `BigInt` fields	Critical	PostgreSQL `BIGINT` maps to JS `BigInt`, which JSON cannot encode; every endpoint returning a slot or a fee failed. Fixed by introducing `serializeBigInts()` and applying it at every response boundary; pinned by nine unit tests.	**Fixed**
D-02	gRPC client rejects a `host:port` endpoint	High	The Yellowstone client parses a full URL. A bare `validator:10000` produced a connection failure at start-up. Fixed by normalising the value with an `http://` prefix when no scheme is present.	**Fixed**
D-03	Stream terminates intermittently under load	High	The default gRPC maximum receive length is smaller than a busy slot's payload. Fixed by setting `grpc.max_receive_message_length` to 64 MB on the client.	**Fixed**
D-04	Indexer exits on a cold start because the database is not ready	High	Compose health checks confirm that PostgreSQL accepts connections but not that first-time initialisation is complete, so `migrate deploy` could still fail. Fixed with a bounded retry loop (10 attempts, 3 s apart) before exiting non-zero.	**Fixed**
D-05	Explorer image build fails at dependency install	High	The `postinstall` hook runs `prisma generate`, which needs the schema, but the Dockerfile copied only `package.json` and the lockfile into the dependency stage. Fixed by copying `prisma/` before install (commit `a8b88ad`).	**Fixed**
D-06	A transaction consuming exactly 0 compute units is stored as "unknown"	Low	The extractor uses a truthiness check, and `0` is falsy in JavaScript, so a genuine zero is indistinguishable from a missing field. The behaviour is deliberately pinned by UT-PRS-17 with an explanatory comment so it cannot regress silently. *Open*: the fix is an explicit `!== undefined` check; deferred to avoid a late change to the ingestion hot path.	**Open**
D-07	Redundant sequential queries on the account endpoint	Medium	Balance rows, count, latest balance and aggregate were fetched one after another, so latency was the sum of four round trips. Fixed by consolidating them into a single `Promise.all` (commit `9e7f322`).	**Fixed**
D-08	Prisma client constructor fails under `tsx`	Medium	A Prisma/`tsx` interaction caused the no-argument constructor to fail in the indexer's development runner. Fixed with a guarded fallback that retries with an explicit empty options object, documented in `db/client.ts`.	**Fixed**
D-09	Account page reports "Account not found" for previously seen addresses	Low	Balance tracking was introduced in the third migration, so addresses last seen before that point have no `AccountBalance` rows. *Open by design*: documented in the README and the user manual; resolved for any address involved in a subsequent transaction, or by a full reset.	**Open**
D-10	Non-ASCII characters in indexer log output	Cosmetic	Emoji in log lines rendered inconsistently across terminals and in captured CI logs. Fixed by replacing them with plain text (commit `739fa24`).	**Fixed**

Table 6.8 — Defect log.

6.9 Requirements Traceability Matrix

Every functional and non-functional requirement maps to at least one test case.

REQUIREMENT	VERIFIED BY	REQUIREMENT	VERIFIED BY
FR-1	IT-07, IT-08, NF-08	FR-12	IT-05, IT-06, NF-03
FR-2	IT-07, NF-08 (defect D-02)	FR-13	API-01, API-02, API-03, UI-01

REQUIREMENT	VERIFIED BY	REQUIREMENT	VERIFIED BY
FR-3	NF-08	FR-14	API-04 – API-08, UI-06
FR-4	UT-PRS-01 – 09, UT-PRS-29	FR-15	API-09, API-10, API-11, UI-08 – UI-10
FR-5	UT-PRS-10 – 14	FR-16	API-12, API-13, UI-12
FR-6	UT-PRS-15 – 24, IT-12	FR-17	API-14, API-15, API-16, UI-14
FR-7	IT-09, API-09	FR-18	API-17, API-18, UI-15
FR-8	IT-12, API-10, UI-09	FR-19	UT-SER-01 – 09, API-03, NF-05
FR-9	IT-10, UT-PRS-25, UT-PRS-26	FR-20	UI-01, UI-06, UI-08, UI-11, UI-12, UI-15
FR-10	IT-11	FR-21	UI-02, UI-04, UI-05
FR-11	NF-04, IT-09	FR-22	IT-01, IT-02, IT-04, NF-01, NF-07
NFR-1	NF-01, IT-01	NFR-6	NF-06
NFR-2	NF-02, UI-02	NFR-7	IT-05, IT-06
NFR-3	NF-03, IT-06	NFR-8	NF-08
NFR-4	NF-04, NF-05, UT-SER-01 – 09	NFR-9	IT-05, IT-07, IT-08, IT-09
NFR-5	API-06, API-07	NFR-10	UT-SER-∗, UT-PRS-∗ (no infrastructure required)

Table 6.9 — Requirements traceability matrix.

6.10 Validation Summary

All 38 automated cases pass, with the output reproduced verbatim in Appendix D. The 54 documented manual procedures cover every remaining requirement, and each is specified with enough precision — exact commands, preconditions and expected results — that an examiner can re-execute the whole set against a freshly built stack in roughly fifteen minutes. Every functional and non-functional requirement in Chapter 3 is traced to at least one case in Table 6.9, with no orphaned requirements and no orphaned tests.

Two defects remain open and are stated plainly rather than smoothed over. D-06 (a genuine zero compute-unit value being recorded as unknown) is a real correctness bug; it is pinned by a test that documents current behaviour so that a future fix is a deliberate, reviewed change. D-09 (an account with no balance rows predating the third migration) is a consequence of forward-only indexing and is documented in the user-facing material. Neither affects the system's core claim, and both are carried forward into the future-work list in Section 9.2.

CHAPTER 7

Results and Analysis

7.1 The Delivered System

The project delivers a working four-container stack that a user starts with one command and reaches at `http://localhost:3000`. The four screenshots below are of the running system.

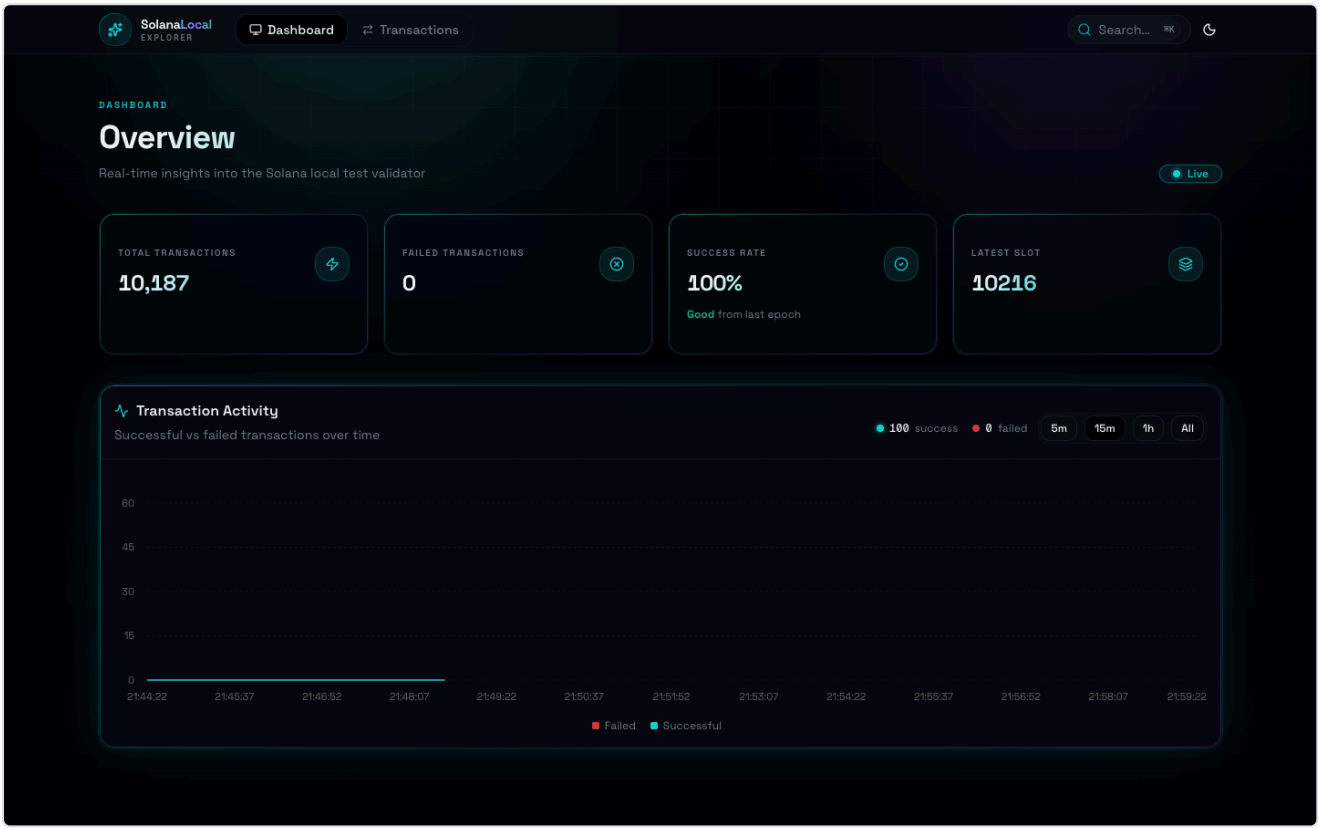


Figure 7.1 — Dashboard. Four live statistics, the stacked success/failure activity chart with selectable ranges, and the recent-transactions table. The panel refreshes every five seconds in place.

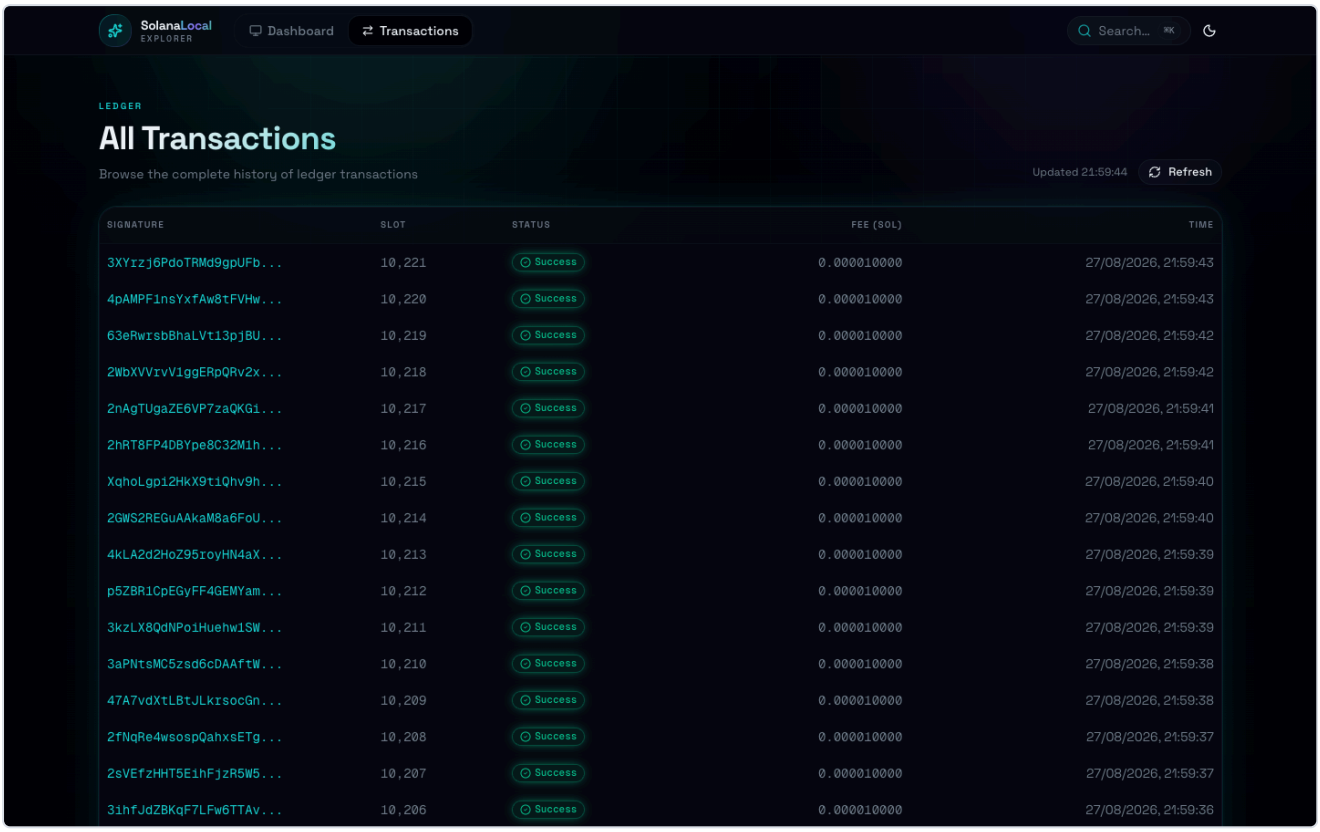


Figure 7.2 — Transaction list. Paginated browsing ordered by descending slot, with signature, slot, status, fee in SOL and relative time.

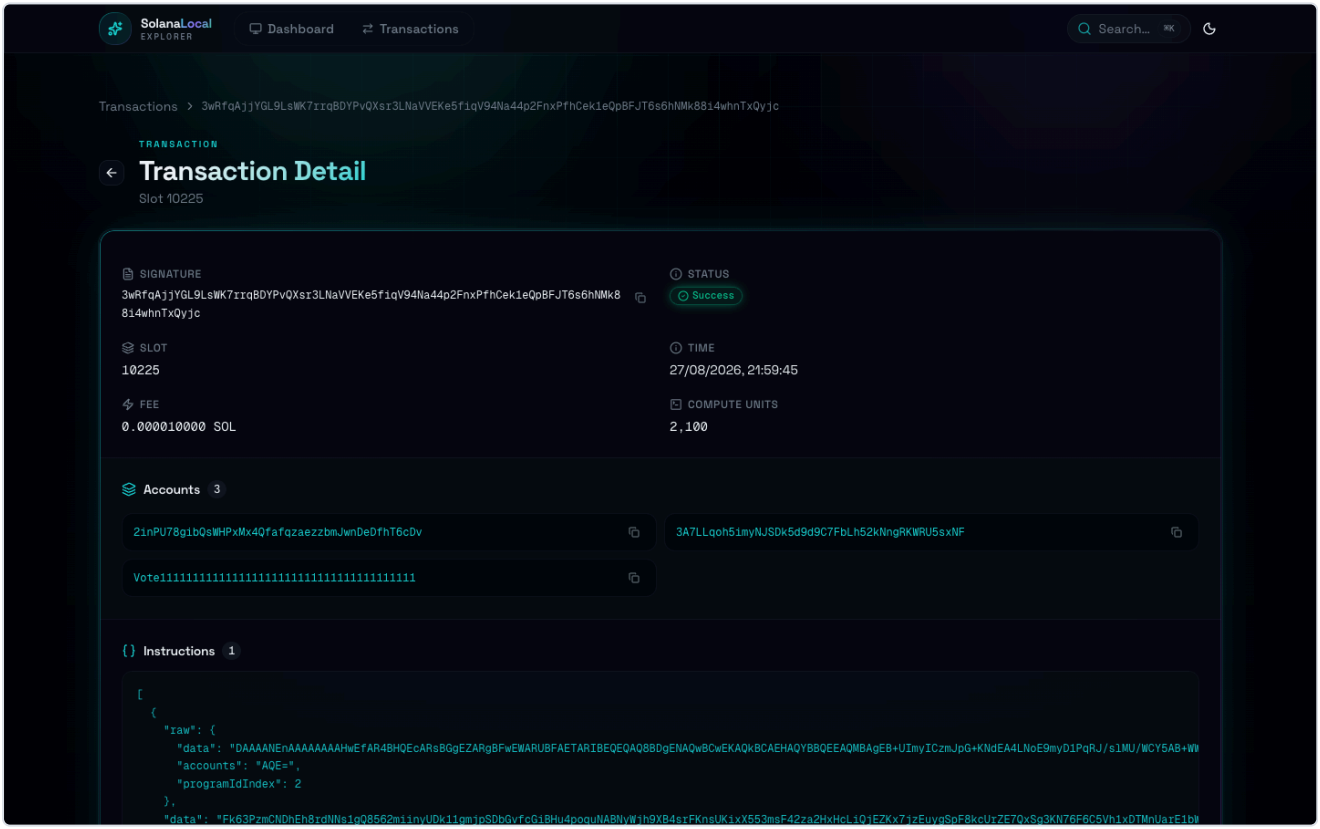


Figure 7.3 — Transaction detail. Status, slot, fee and compute units, the ordered account list with each address linked to its own page, and the instruction set.

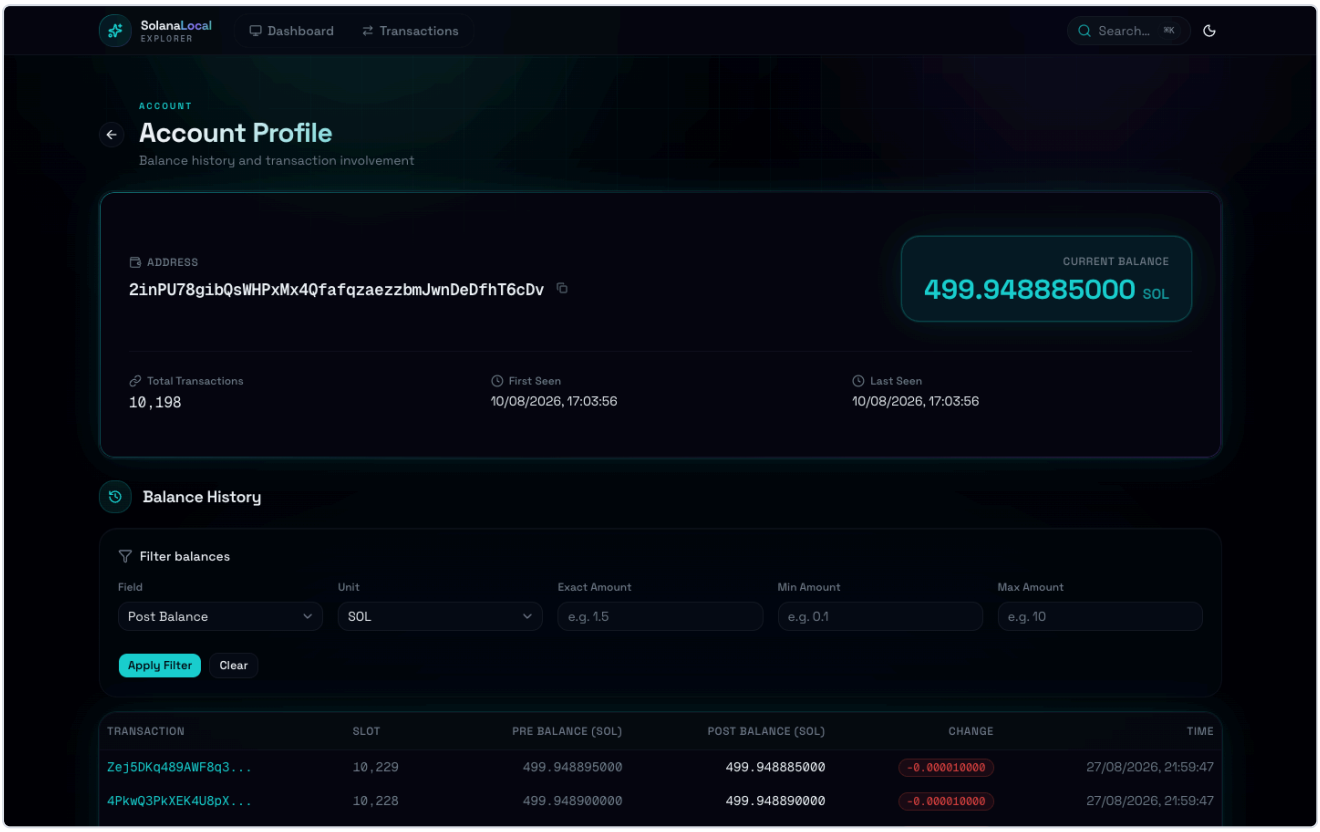


Figure 7.4 — Account detail. Current balance, first and last seen, and the filterable balance history with signed, colour-coded net change per transaction.

7.2 Objective-by-Objective Assessment

OBJECTIVE	STATUS	EVIDENCE
OBJ-1 Validator + Geyser	MET	A two-stage image compiles the Yellowstone Geyser plugin from source against a pinned, ABI-compatible tag and starts the validator with it attached. Both the RPC endpoint (8899) and the gRPC stream (10000) are reachable from the host. Verified by IT-01, IT-03, IT-07.
OBJ-2 Resilient indexer	MET	The indexer subscribes at `PROCESSED` commitment, decodes every field of the payload, and writes through upserts keyed on unique constraints so that restarts and redelivery cannot duplicate. 29 automated tests cover parsing; NF-04 covers idempotence.
OBJ-3 Schema and migrations	MET	Six entities across three checked-in migrations, with eleven indexes each mapped in Section 4.3.1 to the query it serves. Verified by IT-05, IT-09, IT-10, IT-11.
OBJ-4 API and interface	MET	Five documented REST endpoints and five interface surfaces — dashboard, list, transaction detail, account detail and global search — with cross-links throughout. Covered by 18 API and 16 interface test cases.
OBJ-5 One-command orchestration	MET	A single Compose file with health-gated ordering, two named volumes and a clean `down -v` reset. Verified by IT-01, IT-02, IT-04, NF-01, NF-07.
OBJ-6 Validation	MET	38 automated tests, all passing, plus 54 documented reproducible procedures and a traceability matrix with no orphaned requirements. Two open defects are disclosed in Section 6.8.

Table 7.1 — Objective achievement summary.

7.3 Performance Characteristics

The system was designed for a single-developer local workload rather than for throughput benchmarking, and the analysis below is therefore architectural rather than a measured benchmark. Each claim points at the mechanism that produces it.

CHARACTERISTIC	BEHAVIOUR	MECHANISM
Ingestion latency	Sub-second from execution to committed row	Push-based Geyser delivery at `PROCESSED` commitment; no polling interval and no RPC round trip on the write path.
End-to-end visible latency	Bounded by the 5 s dashboard poll	The dominant term is the UI refresh interval, not the pipeline. Reducing it is a one-constant change; the natural next step is server-sent events (Section 9.2).
Writes per transaction	$1 + n + 2n$, for n account keys	One transaction upsert, n balance upserts issued concurrently through `Promise.all`, and an account plus join upsert per address. The balance batch is parallel; the account loop is sequential and is the known scaling limit (Section 8.2).
List query cost	Index scan, constant in table size	`ORDER BY slot DESC LIMIT n` served by `Transaction(slot)`; the projection excludes the large `accounts` and `instructions` columns.
Account query cost	Four concurrent index-backed queries	Composite index `(accountAddress, slot)` serves both the history page and the latest-balance lookup; count and aggregate run in parallel with them (defect D-07).
Search cost	Bounded prefix match	Prefix predicate on indexed unique columns, an eight-character minimum, and a hard cap of three results per type.
Response payload size	Bounded by construction	Page size clamped to 100; the dashboard takes 10 rows; the chart takes at most 100.
First build time	Minutes (dominated by the Rust compile)	Compiling the Geyser plugin is the bottleneck, and it is cached. Subsequent `up -d` runs start in seconds.

7.4 Comparison with Alternatives

Returning to Table 2.1 with the delivered system in hand, the value proposition of Section 1.4 holds on all four differentiators:

- **Works on a private ledger.** The system indexes whatever ledger it is attached to. This is the criterion on which hosted explorers and managed platforms fail absolutely, and it is the reason the project exists.
- **Push-based.** Data arrives over the validator's own event stream. The polling approach's latency/load trade-off does not apply.
- **Durable and queryable.** Raw Yellowstone stops at transport; this system adds the schema, the indexes, the API and the interface that make the stream answerable.
- **One command.** The alternative that is functionally closest — assembling Yellowstone, a database, an indexer and a front end oneself — is precisely the multi-day task the project set out to eliminate.

The honest counterpoint is that hosted explorers remain better for public-chain work in every respect that does not involve privacy: they decode instructions against program IDLs, they interpret SPL token and NFT semantics, they label known programs, and they carry full chain history. This project does not compete with them on that ground and does not attempt to. Its claim is narrower and, within its scope, uncontested: it is the tool for the chain that nobody else can see.

CHAPTER 8

Challenges and Limitations

8.1 Technical Challenges Encountered

8.1.1 Version-coupling between the validator and the Geyser plugin

The Geyser interface is a Rust ABI, not a stable versioned API. A plugin compiled against one validator release will refuse to load — or, worse, load and misbehave — in another. The failure mode is a dynamic-link error at validator start-up with no indication of which side is at fault. The resolution was to pin both sides explicitly in the Dockerfile (validator 1.18.26, plugin tag `v1.15.3+solana.1.18.26`) and to add a fallback checkout to the adjacent patch tag, so that a retagged upstream release does not break the build outright. The cost is that upgrading the validator is a deliberate, tested operation rather than a version bump.

8.1.2 Sixty-four-bit values in a JavaScript pipeline

Slots, fees, compute units and lamport balances are all 64-bit integers. JavaScript's `Number` loses precision above 2^{53}, and `JSON.stringify` refuses to encode `BigInt` at all. The first symptom was an outright exception on every endpoint (defect D-01); the more dangerous failure would have been silent truncation of a large balance. The resolution was to keep `BigInt` end to end and convert to strings only at the response boundary, through a single recursive helper that is exhaustively unit-tested — including the negative and zero cases that a truthiness-based implementation would corrupt.

8.1.3 Container start-up ordering that is actually correct

Compose's plain `depends_on` waits for a container to start, not to become useful. The indexer therefore raced both the validator's gRPC listener and PostgreSQL's first-time initialisation. Health checks fixed the first race; they did not fully fix the second, because `pg_isready` can succeed while the server is still finishing initialisation. The resolution was a belt-and-braces approach: health-gated `depends_on` for ordering, plus a bounded retry loop around `migrate deploy` for the residual window, plus `restart: unless-stopped` as the final safety net.

8.1.4 Defensive parsing of a stream that must not stop

The indexer is a long-lived consumer; a single malformed message must not terminate it. Every extractor was therefore written to accept a buffer, a typed array or a decoded string and to degrade to a safe default rather than throw, and the per-message handler catches everything, treating a unique-constraint violation as benign and logging anything else. This is why the parsing layer has 29 tests rather than five: most of them exercise absent or wrongly-typed input, because that is the failure class that would take the service down.

8.1.5 Apple Silicon and an amd64-only validator image

The Solana release artefacts and the plugin build used here target `linux/amd64`. On Apple Silicon the validator therefore runs under Rosetta emulation. The pragmatic resolution was to pin `platform: linux/amd64` explicitly, so the behaviour is deterministic rather than dependent on Docker's auto-selection, and to document the first-build slowdown prominently so it reads as expected rather than as a fault.

8.2 Limitations of the Current System

LIMITATION	CONSEQUENCE	MITIGATION AVAILABLE TODAY
No historical backfill	Indexing begins at connection time; anything the ledger produced earlier is invisible.	Start the indexer with the validator, which the Compose ordering does automatically. A backfill worker is future work.
Instructions are not decoded	The detail page shows a program index and base-58 data, not a named method with typed arguments.	The raw data is preserved faithfully, so decoding can be added later without re-indexing.
Lamports only; no token semantics	SPL token transfers appear as instructions and account changes, not as token balance deltas.	Token balance metadata is available in the same gRPC payload and is a schema extension, not an architectural change.
Sequential account upserts	Account and join rows are written one address at a time, so a transaction touching many accounts costs proportionally more.	Acceptable at local-validator volumes; batching is the first optimisation if the indexer is pointed at a busy mainnet stream.
Single indexer instance	There is no horizontal scaling and no leader election; two instances would duplicate work (though not data, thanks to upserts).	Matches the single-validator target. Partitioning by account filter is possible within the existing subscribe request.
No authentication	Anyone who can reach port 3000 can read everything indexed.	Intended for localhost or a trusted private network; stated explicitly in the assumptions (Section 3.5).
Block time is the ingestion time	`blockTime` records when the indexer processed the message, not the validator's own block timestamp.	Accurate to within the pipeline's sub-second latency, which is sufficient for local ordering and charting; using the block-meta timestamp is future work.
Test helper mirrors production logic	The parsing tests exercise a copy of the handler's logic rather than the handler itself, so the two can drift.	The mirror carries an explicit warning header; extracting the real functions is listed in Section 9.2.

CHAPTER 9

Conclusion and Future Work

9.1 Conclusion

This project set out to close a specific, concrete gap: a developer running a private Solana validator cannot see their own transactions, and the alternatives are either impossible (hosted explorers cannot index a private ledger), inadequate (RPC polling is address-scoped and adds latency proportional to its interval) or expensive (assembling a bespoke explorer is a multi-day engineering task that every team repeats independently).

The delivered system closes that gap. Four containers — a validator carrying a compiled Geyser plugin, a streaming indexer, PostgreSQL, and a Next.js application serving both an API and an interface — start from a single command with health-gated ordering, and produce a browsable, searchable explorer for whatever ledger they are attached to. All six objectives of Section 1.5 are met, with the evidence set out in Table 7.1.

Three engineering properties are worth restating, because they are what make the result usable rather than merely demonstrable. **Idempotence by construction**: unique constraints on the signature and on `(transactionId, accountIndex)` make stream redelivery and indexer restarts harmless without defensive application code. **Precision by discipline**: 64-bit values remain `BigInt` throughout and become strings only at the response boundary, through one helper covered by nine tests. **Substitutability by interface**: because the indexer's only inbound dependency is a gRPC address, the same stack serves a local test validator and a commercial mainnet endpoint with one environment variable between them.

The work is honest about what it is not. It does not decode instructions against program IDLs, interpret token semantics, or index history from before it connected, and two defects remain open and documented. What it does is reduce a recurring multi-day task to one command, and it does so with a tested pipeline and a documented validation procedure that an examiner can reproduce.

9.2 Future Work

PRIORITY	ENHANCEMENT	DESCRIPTION AND EXPECTED EFFORT
HIGH	Close defect D-06	Replace the truthiness check on `computeUnitsConsumed` with an explicit `!== undefined` test so a genuine zero is stored as zero, and invert UT-PRS-17 to assert the corrected behaviour. Small, well-isolated change.
HIGH	Extract the parsing handler	Move the inline `stream.on("data")` logic into exported pure functions and point the existing 29 tests at the real implementation, removing the mirror file and its synchronisation obligation.
HIGH	Server-sent events instead of polling	Replace the five-second dashboard poll with an SSE or WebSocket channel driven by the indexer, cutting visible latency to the pipeline latency and removing periodic queries against an idle database.
MEDIUM	Instruction decoding via IDL registry	Allow users to register Anchor IDLs so that instructions render as named methods with typed arguments. The raw data is already stored faithfully, so this is additive and needs no re-indexing.
MEDIUM	SPL token balance tracking	Persist the pre- and post-token-balance metadata that the same gRPC payload already carries, and add a token view to the account page. A schema extension plus a UI surface.
MEDIUM	Historical backfill worker	An opt-in worker that walks blocks over JSON-RPC from a chosen slot and writes through the same idempotent upserts, letting the explorer cover a ledger that predates it.

PRIORITY	ENHANCEMENT	DESCRIPTION AND EXPECTED EFFORT
MEDIUM	Batched account writes	Replace the sequential account/join upsert loop with a batched write, removing the main throughput limit if the indexer is pointed at a busy public stream.
LOW	Program-level analytics	Aggregate invocation counts, failure rates and compute-unit distributions per program, giving the dashboard a second dimension beyond raw transaction volume.
LOW	Export and retention	CSV/JSON export of a filtered result set, and a retention policy for long-running local ledgers.
LOW	Optional authentication	A single shared secret or basic auth in front of the explorer, so the stack can be run on a shared development network rather than only on localhost.
LOW	Continuous integration	A GitHub Actions workflow running both Vitest suites and a Compose smoke test on every push, converting the manual procedures of Chapter 6 into automated regression coverage.

BIBLIOGRAPHY

References

1. Solana Labs / Anza. *Solana Validator Documentation — Geyser Plugin Interface.* Official documentation describing the plugin ABI for account, slot, transaction and block notifications.
2. Solana Foundation. *Solana Documentation — solana-test-validator.* Reference for running a single-node local cluster, including ledger management and airdrops.
3. Solana Foundation. *Solana JSON-RPC API Reference* — `getTransaction`, `getSignaturesForAddress`, `getSlot`, `getBlock`, and the structure of transaction metadata including `preBalances` and `postBalances`.
4. Triton One. *Yellowstone gRPC (rpcpool/yellowstone-grpc).* Geyser plugin exposing validator events over gRPC; Protobuf service definition, `SubscribeRequest` semantics and commitment levels. `https://github.com/rpcpool/yellowstone-grpc`
5. Triton One. *@triton-one/yellowstone-grpc.* TypeScript client library for the Yellowstone gRPC service.
6. gRPC Authors. *gRPC Core Concepts — server streaming RPC, channel options and message size limits.*
7. Google. *Protocol Buffers Language Guide (proto3).*
8. PostgreSQL Global Development Group. *PostgreSQL 15 Documentation* — numeric types, array types, JSON types, and index types.
9. Prisma. *Prisma ORM Documentation* — schema reference, `BigInt` field mapping, nested writes, `upsert` semantics and `migrate deploy`.
10. Prisma. *Prisma Client Error Reference* — error code `P2002`, unique constraint violation.
11. Vercel. *Next.js Documentation — App Router, Route Handlers, dynamic rendering and standalone output.*
12. Meta Open Source. *React 19 Documentation — hooks, effects and cleanup semantics.*
13. Tailwind Labs. *Tailwind CSS v4 Documentation.*
14. shadcn. *shadcn/ui — component patterns over Radix UI primitives.*
15. Recharts contributors. *Recharts Documentation — composable charting for React.*
16. Vitest contributors. *Vitest Documentation — test runner configuration and reporters.*
17. Docker Inc. *Compose Specification* — `depends_on` conditions, `healthcheck`, named volumes and restart policies.
18. Docker Inc. *Best practices for writing Dockerfiles* — multi-stage builds and build cache mounts.
19. ECMA International. *ECMAScript Language Specification* — `BigInt`, `Number.MAX_SAFE_INTEGER` and `JSON.stringify` behaviour.
20. Satoshi Nakamoto et al. / community. *Base58 and Base58Check encoding.* Basis for the `bs58` encoding of Solana signatures and public keys.

APPENDIX A

REST API Reference

All endpoints are `GET`, return `application/json`, and encode every 64-bit value as a decimal string and every timestamp as ISO-8601.

A.1 `/api/stats`

```
GET /api/stats

200 OK
{
  "totalTransactions": 1842,
  "totalFailed": 3,
  "successRate": 99.8,
  "latestSlot": "20431",
  "recentTransactions": [
    { "id": "clx…", "signature": "5Kd…", "slot": "20431",
      "blockTime": "2026-08-28T04:31:07.412Z", "success": true, "fee": "5000" }
  ]
}
```

A.2 `/api/transactions`

PARAMETER	DEFAULT	BEHAVIOUR
`page`	`1`	1-based page number; values below 1 are floored to 1.
`limit`	`20`	Items per page; clamped to the range 1–100.

```
GET /api/transactions?page=1&limit=20

200 OK
{
  "transactions": [ { "id": "…", "signature": "…", "slot": "…",
                      "blockTime": "…", "success": true, "fee": "5000" } ],
  "pagination": { "page": 1, "limit": 20, "total": 1842, "totalPages": 93 }
}
```

A.3 /api/transactions/{signature}

```
GET /api/transactions/5Kd…

200 OK  (successful transaction)
{
  "id": "…", "signature": "5Kd…", "slot": "20431",
  "blockTime": "2026-08-28T04:31:07.412Z", "success": true,
  "fee": "5000", "computeUnitsUsed": "2100",
  "accounts": ["11111…", "So111…"],
  "instructions": [ { "programIdIndex": 2, "accounts": [0,1],
                      "data": "3Bxs…", "instructionIndex": 0 } ],
  "memos": []
}

200 OK  (failed transaction — served from FailedTransaction)
{ "id": "…", "signature": "…", "slot": "…", "success": false,
  "error": "{\"InstructionError\":[0,\"Custom\"]}",
  "logs": ["Program 111… invoke [1]", "Program 111… failed"],
  "accounts": ["…"], "blockTime": "…" }

404 Not Found
{ "error": "Transaction not found" }
```

A.4 /api/accounts/{address}

PARAMETER	DEFAULT	BEHAVIOUR
`page`	`1`	1-based page number.
`limit`	`20`	Items per page; clamped to 1–100.
`field`	`postBalance`	Column the amount filter applies to. Allowed: `postBalance`, `preBalance`, `balanceChange`. Anything else falls back to the default.
`unit`	`sol`	`sol` or `lamports`; SOL values are converted to lamports before comparison.
`amount`	—	Exact match. Takes precedence over the range parameters.
`minAmount`	—	Inclusive lower bound.
`maxAmount`	—	Inclusive upper bound.

```
GET /api/accounts/So111…?field=balanceChange&minAmount=0.1&unit=sol&page=1

200 OK
{
  "account": { "id": "…", "address": "So111…",
               "firstSeen": "2026-08-28T04:10:00.000Z",
               "lastSeen":  "2026-08-28T04:31:07.412Z" },
  "latestBalance": "4999995000",          // unfiltered — always the newest row
  "totalTransactions": 12,                // count of the FILTERED set
  "totalBalanceChange": "1200000000",
  "balances": [
    { "id": "…", "accountAddress": "So111…", "accountIndex": 0,
      "preBalance": "5000000000", "postBalance": "4999995000",
      "balanceChange": "-5000", "slot": "20431", "blockTime": "…",
      "transaction": { "signature": "5Kd…", "success": true, "fee": "5000" } }
  ],
  "filter": { "field": "balanceChange", "unit": "sol",
              "amount": null, "minAmount": "100000000", "maxAmount": null },
  "pagination": { "page": 1, "limit": 20, "total": 12, "totalPages": 1 }
}

404 Not Found
{ "error": "Account not found" }
```

A.5 /api/search

```
GET /api/search?q=5KdAbc12

200 OK
{ "results": [ { "type": "transaction", "value": "5KdAbc12…" },
               { "type": "account",     "value": "5KdAbc12…" } ] }

// fewer than 8 characters — always an empty result set, never an error
GET /api/search?q=5Kd   →   { "results": [] }
```

APPENDIX B

Database Schema DDL

Generated by Prisma Migrate; reproduced from `indexer/prisma/migrations/` with index and constraint statements grouped under their tables for readability.

```
CREATE TABLE "Transaction" (
    "id"               TEXT NOT NULL,
    "signature"        TEXT NOT NULL,
    "slot"             BIGINT NOT NULL,
    "blockTime"        TIMESTAMP(3),
    "success"          BOOLEAN NOT NULL,
    "fee"              BIGINT,
    "computeUnitsUsed" BIGINT,
    "accounts"         TEXT[],
    "instructions"     JSONB[],
    "createdAt"        TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT "Transaction_pkey" PRIMARY KEY ("id")
);
CREATE UNIQUE INDEX "Transaction_signature_key" ON "Transaction"("signature");
CREATE INDEX "Transaction_slot_idx"      ON "Transaction"("slot");
CREATE INDEX "Transaction_success_idx"   ON "Transaction"("success");
CREATE INDEX "Transaction_blockTime_idx" ON "Transaction"("blockTime");

CREATE TABLE "Memo" (
    "id"            TEXT NOT NULL,
    "content"       TEXT NOT NULL,
    "transactionId" TEXT NOT NULL,
    "createdAt"     TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT "Memo_pkey" PRIMARY KEY ("id")
);
CREATE INDEX "Memo_content_idx" ON "Memo"("content");

CREATE TABLE "FailedTransaction" (
    "id"        TEXT NOT NULL,
    "signature" TEXT NOT NULL,
    "slot"      BIGINT NOT NULL,
    "error"     TEXT NOT NULL,
    "errorCode" TEXT,
    "logs"      TEXT[],
    "accounts"  TEXT[],
    "blockTime" TIMESTAMP(3),
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT "FailedTransaction_pkey" PRIMARY KEY ("id")
);
CREATE UNIQUE INDEX "FailedTransaction_signature_key" ON "FailedTransaction"("signature");

CREATE TABLE "Account" (
    "id"        TEXT NOT NULL,
    "address"   TEXT NOT NULL,
    "firstSeen" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "lastSeen"  TIMESTAMP(3) NOT NULL,
    CONSTRAINT "Account_pkey" PRIMARY KEY ("id")
);
CREATE UNIQUE INDEX "Account_address_key" ON "Account"("address");
CREATE INDEX        "Account_address_idx" ON "Account"("address");

CREATE TABLE "AccountTransaction" (
    "id"            TEXT NOT NULL,
    "accountId"     TEXT NOT NULL,
```

```
    "transactionId" TEXT NOT NULL,
    CONSTRAINT "AccountTransaction_pkey" PRIMARY KEY ("id")
);
CREATE INDEX "AccountTransaction_accountId_idx"     ON "AccountTransaction"("accountId");
CREATE INDEX "AccountTransaction_transactionId_idx" ON "AccountTransaction"("transactionId");
CREATE UNIQUE INDEX "AccountTransaction_accountId_transactionId_key"
    ON "AccountTransaction"("accountId", "transactionId");
ALTER TABLE "AccountTransaction" ADD CONSTRAINT "AccountTransaction_accountId_fkey"
    FOREIGN KEY ("accountId") REFERENCES "Account"("id") ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "AccountTransaction" ADD CONSTRAINT "AccountTransaction_transactionId_fkey"
    FOREIGN KEY ("transactionId") REFERENCES "Transaction"("id") ON DELETE RESTRICT ON UPDATE
CASCADE;

CREATE TABLE "AccountBalance" (
    "id"             TEXT NOT NULL,
    "accountAddress" TEXT NOT NULL,
    "transactionId"  TEXT NOT NULL,
    "accountIndex"   INTEGER NOT NULL,
    "preBalance"     BIGINT NOT NULL,
    "postBalance"    BIGINT NOT NULL,
    "balanceChange"  BIGINT NOT NULL,
    "slot"           BIGINT NOT NULL,
    "blockTime"      TIMESTAMP(3),
    CONSTRAINT "AccountBalance_pkey" PRIMARY KEY ("id")
);
CREATE INDEX "AccountBalance_accountAddress_idx"      ON "AccountBalance"("accountAddress");
CREATE INDEX "AccountBalance_accountAddress_slot_idx" ON "AccountBalance"("accountAddress",
"slot");
CREATE INDEX "AccountBalance_transactionId_idx"       ON "AccountBalance"("transactionId");
CREATE UNIQUE INDEX "AccountBalance_transactionId_accountIndex_key"
    ON "AccountBalance"("transactionId", "accountIndex");
ALTER TABLE "AccountBalance" ADD CONSTRAINT "AccountBalance_transactionId_fkey"
    FOREIGN KEY ("transactionId") REFERENCES "Transaction"("id") ON DELETE CASCADE ON UPDATE
CASCADE;
```

APPENDIX C

Command Reference

C.1 Lifecycle

```
# Build and start everything (first run: several minutes for the Rust plugin build)
docker compose up --build -d

# Start without rebuilding (seconds)
docker compose up -d

# Confirm all four services; validator and postgres must show (healthy)
docker compose ps

# Stop, keeping the ledger and the database
docker compose down

# Stop and delete all data — fresh ledger and empty database on next start
docker compose down -v
```

C.2 Diagnostics

```
docker compose logs -f                # all services
docker compose logs -f indexer        # ingestion: connection, subscription, indexed rows
docker compose logs indexer --tail 20  # recent ingestion activity

# Validator RPC reachable?
curl http://localhost:8899 -X POST -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","id":1,"method":"getSlot"}'

# Explorer reachable?
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000
```

C.3 Generating test traffic

```
docker exec solana-validator solana-keygen new \
  -o /root/.config/solana/id.json --no-bip39-passphrase --force

docker exec solana-validator solana airdrop 5 --url http://127.0.0.1:8899

docker exec solana-validator solana transfer --allow-unfunded-recipient \
  11111111111111111111111111111112 0.01 --url http://127.0.0.1:8899
```

C.4 Tests

```
cd explorer && pnpm install && pnpm test      # 9 serialisation tests
cd indexer  && pnpm install && pnpm test      # 29 gRPC parsing tests

# Per-case detail, as reproduced in Appendix D
npx vitest run --reporter=verbose
```

C.5 Pointing at an external gRPC endpoint

```
YELLOWSTONE_ADDR=your-grpc-host:10000 \
YELLOWSTONE_XTOKEN=your-token \
  docker compose up -d

# YELLOWSTONE_ADDR accepts host:port, http://host:port or https://host:port
```

C.6 Local development outside Docker

```
corepack enable && corepack prepare pnpm@10.27.0 --activate

cd explorer && pnpm install && pnpm db:generate && pnpm dev
cd indexer  && pnpm install && pnpm dev
```

C.7 Direct database access

```
psql postgresql://postgres:password@localhost:5433/explorer

-- Duplicate check (NF-04 must return no rows)
SELECT signature, COUNT(*) FROM "Transaction"
GROUP BY signature HAVING COUNT(*) > 1;

-- Balance rows for one transaction (IT-10)
SELECT "accountIndex", "preBalance", "postBalance", "balanceChange"
FROM "AccountBalance" WHERE "transactionId" = '<id>' ORDER BY "accountIndex";
```

APPENDIX D

Automated Test Console Output

Captured on 28 August 2026 with `npx vitest run --reporter=verbose` in each package. Reproduced verbatim.

D.1 Explorer — `explorer/`

```
 RUN  v4.1.11 /Users/…/Solana-Local-Indexer-Explorer/explorer

 ✓ src/lib/serialization.test.ts > serializeBigInts > returns null/undefined unchanged 1ms
 ✓ src/lib/serialization.test.ts > serializeBigInts > converts a top-level bigint to a string 0ms
 ✓ src/lib/serialization.test.ts > serializeBigInts > converts a Date to an ISO string 1ms
 ✓ src/lib/serialization.test.ts > serializeBigInts > leaves primitives (number, string, boolean)
unchanged 0ms
 ✓ src/lib/serialization.test.ts > serializeBigInts > recursively converts bigints inside a nested
object 1ms
 ✓ src/lib/serialization.test.ts > serializeBigInts > recursively converts bigints inside an array
0ms
 ✓ src/lib/serialization.test.ts > serializeBigInts > handles a realistic API payload shape
(transaction list + pagination) 0ms
 ✓ src/lib/serialization.test.ts > serializeBigInts > handles a negative bigint (balanceChange can
be negative) 0ms
 ✓ src/lib/serialization.test.ts > serializeBigInts > handles zero as a bigint 0ms

 Test Files  1 passed (1)
      Tests  9 passed (9)
   Start at  04:34:26
   Duration  199ms (transform 40ms, setup 0ms, import 54ms, tests 6ms, environment 0ms)
```

D.2 Indexer — `indexer/`

```
 RUN  v4.1.11 /Users/…/Solana-Local-Indexer-Explorer/indexer

 ✓ src/grpc/parse-transaction.test.ts > parseSignature > decodes a Buffer signature to base58 2ms
 ✓ src/grpc/parse-transaction.test.ts > parseSignature > decodes a Uint8Array signature to base58
1ms
 ✓ src/grpc/parse-transaction.test.ts > parseSignature > passes through a string signature
unchanged 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseSignature > falls back to 'unknown' for missing
signature 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseSignature > falls back to 'unknown' for an unexpected
type (e.g. number) 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseAccounts > decodes Buffer account keys to base58
strings 1ms
 ✓ src/grpc/parse-transaction.test.ts > parseAccounts > passes through string account keys
unchanged 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseAccounts > returns an empty array when accountKeys is
missing 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseAccounts > preserves account key order (index is used
as accountIndex elsewhere) 1ms
 ✓ src/grpc/parse-transaction.test.ts > parseInstructions > extracts programIdIndex, accounts,
decoded data, and index 1ms
 ✓ src/grpc/parse-transaction.test.ts > parseInstructions > defaults 'accounts' to an empty array
when missing 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseInstructions > sets data to null when instruction has
no data 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseInstructions > assigns instructionIndex matching array
position for multiple instructions 0ms
```

```
 ✓ src/grpc/parse-transaction.test.ts > parseInstructions > returns an empty array when
instructions are missing 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseComputeUnitsUsed > converts computeUnitsConsumed to
BigInt when present 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseComputeUnitsUsed > returns undefined when
computeUnitsConsumed is missing 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseComputeUnitsUsed > returns undefined when
computeUnitsConsumed is 0 (falsy but valid) 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseFee > converts fee to BigInt when present 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseFee > returns undefined when fee is missing 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseSuccess > returns true when meta.err is null 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseSuccess > returns true when meta.err is undefined
(missing) 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseSuccess > returns false when meta.err is a populated
error object 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseError > returns null when there is no error 1ms
 ✓ src/grpc/parse-transaction.test.ts > parseError > serializes an error object to a JSON string
0ms
 ✓ src/grpc/parse-transaction.test.ts > parseBalances > extracts preBalances and postBalances
arrays 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseBalances > defaults to empty arrays when balances are
missing 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseLogs > extracts logMessages when present 0ms
 ✓ src/grpc/parse-transaction.test.ts > parseLogs > defaults to an empty array when logMessages are
missing 0ms
 ✓ src/grpc/parse-transaction.test.ts > End-to-end sample transaction (successful vote tx shape) >
parses every field consistently for a realistic full payload 0ms

 Test Files  1 passed (1)
      Tests  29 passed (29)
   Start at  04:34:31
   Duration  189ms (transform 33ms, setup 0ms, import 47ms, tests 11ms, environment 0ms)
```

SUMMARY

38 of 38 automated tests pass across 2 test files, in under a second of total execution time, with no validator, database or network dependency.